Pure Insights Command Center

An operational admin dashboard for the custom analytics portal that replaced Tableau Server. It provides role-based monitoring of system health and orphaned reports, real-time WebSocket alerts, and Snowflake credit attribution down to unit cost per active user view.

GITHUB REPOSITORY (PUBLIC)

https://github.com/tridibjena/pure-insights-command-center

Branch `main` · commit `4b1424d` · clone with `git clone https://github.com/tridibjena/pure-insights-command-center.git`

1 **Runnable full-stack codebase**

React 19 frontend and Node.js/Express 5 backend over PostgreSQL 16, with a Python data pipeline that loads the CSV and cleans its anomalies. One-command setup, 107 automated tests.

`Repository` `Quick start` `server/` `client/` `Part 1 of this document`

2 **REST API & WebSocket technical specification**

Auth and sessions, RBAC matrix, every REST endpoint, alerting rules, incident lifecycle, and the WebSocket handshake, messages and reconnect protocol. Machine-readable OpenAPI 3.1 and AsyncAPI 3.0 versions are included.

`docs/API_SPEC.md` `openapi.yaml` `asyncapi.yaml` `Part 2`

3 **Snowflake cloud cost allocation SQL script suite**

Ten scripts (00–09) over simulated ACCOUNT_USAGE logs. They attribute credits to individual reports two ways, compute unit cost per active user view, and finish with six reconciliation checks that all pass.

`sql/snowflake/` `Methodology` `Part 3` `Appendix: full SQL`

Stack	Submitted by	Dataset	Date
CSV → PostgreSQL → Node.js/Express → React	github.com/tridibjena	Case_Study_3_Pure_Insights_Monitoring.csv (2,000 rows)	11 October 2026

CONTENTS

Submission overview

0	Tasks, anomalies and verification at a glance	Compliance with the brief
1	Deliverable 1: Runnable full-stack codebase	Architecture, quick start, screenshots
2	Deliverable 2: REST API & WebSocket specification	Full specification
3	Deliverable 3: Snowflake cost allocation SQL suite	Suite, methodology, results
4	Key findings and assumptions	Business synthesis
A	Appendix: SQL scripts 00–09 in full	Source listings

Tasks → implementation

Requirement from the brief	How it is met	Where
Task 1: React + Node.js/Express dashboard with RBAC, monitoring system health and orphaned reports	Pages for System Health, Reports (with 20-snapshot drill-down), Orphaned Reports (with audited admin reassignment), Alerts, Cost Allocation, Data Quality and Administration. Four user roles plus a service role, enforced on every route and every WebSocket message; Report Owners are row-scoped.	server/, client/, RBAC.md
Task 2: WebSocket / SSE alerts when load latency exceeds 15 s or automated refreshes fail	WebSocket channel at `/ws`, authenticated with the session cookie. `LATENCY_SLO_BREACH` fires when latency is strictly above 15 s; `REFRESH_FAILURE` fires on FAILED / TIMED_OUT. One incident per report and rule, with escalation and auto-resolve on recovery. A replayer streams the CSV through the live ingest path.	server/src/services/, API_SPEC.md §5–6
Task 3: SQL against simulated Snowflake ACCOUNT_USAGE logs, credits per report, unit cost per active user view	Simulated QUERY_HISTORY, QUERY_ATTRIBUTION_HISTORY and WAREHOUSE_METERING_HISTORY that reconcile to the CSV. Method A (QUERY_TAG attribution) and Method B (metered-bill allocation) are both computed. Unit cost is $2.434 Snowflake + $0.307 AWS = **$2.740 per active view** at $3/credit.	sql/snowflake/, Methodology

Injected data anomalies: identified, cleaned and documented

Anomaly (brief)	Found in the CSV	Treatment
Orphaned owners (NaN / 'ORPHANED_REPORT')	245 rows (128 nan, 117 ORPHANED_REPORT); 14 reports orphaned in the latest snapshot	Owner nulled and flagged; Orphaned Reports page; audited reassignment; $480.9K unowned spend surfaced
Negative Snowflake credits	24 rows, −13.6 to −0.8 (total −161.04)	Quarantined in the pipeline and in SQL (not abs()'d, not zeroed); clean total 1,206,352.48 credits
Latency spikes up to 55 s	Max 55.0 s (6 readings exactly at 55.0); 1,490 rows (74.5 %) above 15 s; 7 rows at exactly 15.0 s	The 15 s threshold triggers alerts; 55 s is the observed extreme, kept unchanged and alerted on as CRITICAL. Exactly 15.0 s does not alert
Active users vs AWS cost discrepancies	r = −0.027; 4 zero-view months with cost; 93 rows with peak users > views; 135 rows above the 3×IQR cost-per-view fence ($2.06)	Flagged by four rules, analysed in `07_aws_cost_vs_usage_discrepancy.sql`, plotted on the Cost page

The register lives in quality_rules.py and is written out to docs/DATA_QUALITY.md. Raw values are never deleted: they stay in the `raw` schema and every finding is logged per row and rule.

Verification

Check	Command	Result
Automated tests	`npm test`	107 passing: pytest 52, server node:test 39, client Vitest 16
SQL reconciliation (PostgreSQL)	`npm run verify`	All 6 checks PASS
Snowflake dialect	`npm run verify:snowflake`	Every statement parses as Snowflake SQL; the suite executes on a Snowflake emulator with identical figures
API contract	`openapi.contract.test.js`	Every mounted Express route is documented in openapi.yaml, and every documented route exists
Lint	`npm run lint`	ruff + eslint clean

DELIVERABLE 1

Runnable full-stack codebase

The GitHub repository with the React frontend and Node.js/Express backend, on PostgreSQL, set up with one command.

`github.com/tridibjena/pure-insights-command-center` `README.md` `docs/ARCHITECTURE.md`

Quick start

Prerequisites: Docker (Desktop running), Node.js ≥ 20, Python ≥ 3.11.

```
cp .env.example .env          # local defaults; change JWT_SECRET for anything beyond local use
npm install
npm run setup                 # Postgres (docker compose) → Python venv → pipeline → demo users
npm run dev                   # API on :4000, dashboard on http://localhost:5173
```

Sign in with any demo account. The password is `SEED_DEMO_PASSWORD` from `.env` (default `PureInsights!2026`).

Account	Role	Sees
`admin@pureinsights.local`	Administrator	everything, plus owner reassignment, accounts and the audit log
`sre@pureinsights.local`	Platform Engineer	health, alerts (ack/resolve), reports, orphans, data quality, replay control
`finops@pureinsights.local`	FinOps Analyst	cost allocation, chargeback, data quality, read-only health and alerts
`user_7@company.com`	Report Owner	only the 7 reports user_7 currently owns, and their alerts

To see the real-time alerting: sign in as `sre@…` or `admin@…`, open **Alerts**, and click **Start replay**. The 2,000 CSV observations stream through the live ingest → alert → WebSocket path, one snapshot at a time.

Command	What it does
`npm run setup`	Starts Postgres, creates the Python venv, runs the full pipeline, seeds users (idempotent; safe to re-run)
`npm run pipeline`	migrate → ingest → clean → forensic checks → simulate ACCOUNT_USAGE → cost SQL suite → verify → export → Snowflake-dialect check (emulator) → DQ report → findings
`npm run dev`	API (`node --watch`) and Vite dev server

Command	What it does
`npm start`	Production build: Express serves the built React app on :4000
`npm test`	pytest (52) + server `node:test` (39) + client Vitest (16)
`npm run verify`	Rebuilds the `cost.*` views from `sql/snowflake/01-09` and asserts all reconciliation checks pass
`npm run verify:snowflake`	Parses the suite as Snowflake SQL and executes it on the Snowflake emulator; checks must match PostgreSQL
`npm run lint`	ruff + eslint (server and client)
`npm run screenshots`	Re-captures the README screenshots from the running app (Playwright; first run `pipeline/.venv/bin/playwright install chromium`)

Repository layout

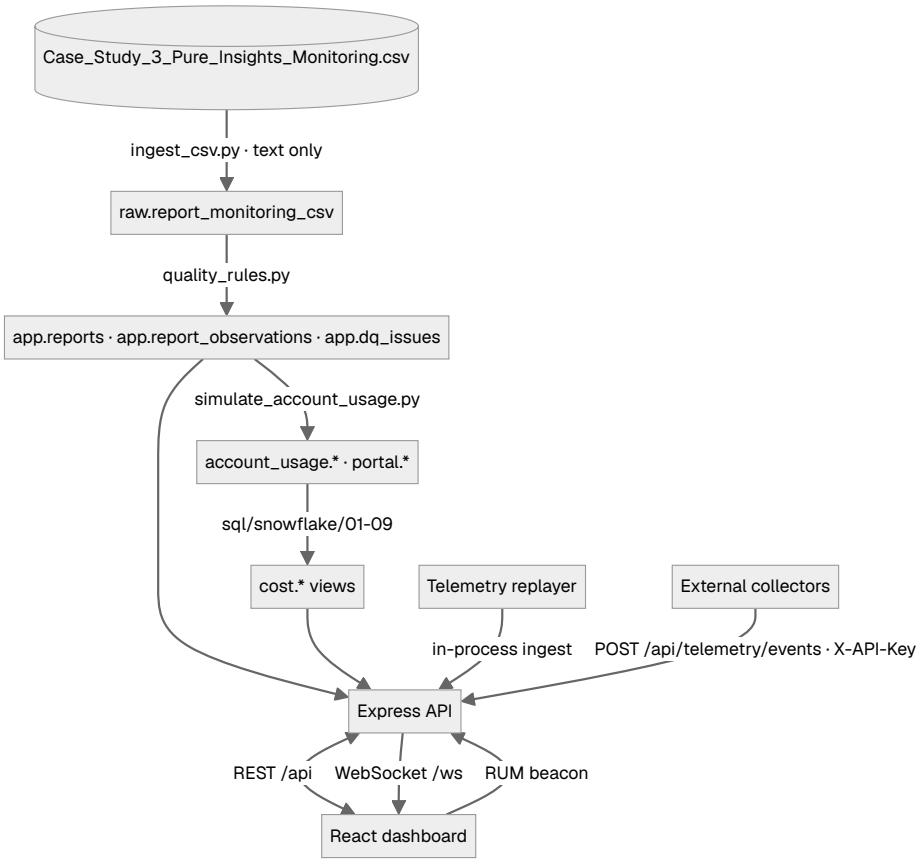
```
├─ data/raw/                 source CSV (data/simulated/ is generated)
├─ db/migrations/            idempotent PostgreSQL DDL
├─ pipeline/                 Python: ingest · quality rules · simulation · SQL-suite runner · docs
generators · screenshots
├─ sql/snowflake/            Deliverable 3 — cost allocation SQL suite (00-09)
├─ server/                   Node.js/Express API, RBAC, alerting, WebSocket hub, replay (+ tests)
├─ client/                   React dashboard (+ tests)
└─ docs/                     API spec (MD + OpenAPI + AsyncAPI), architecture, RBAC, methodology,
                             generated data-quality and findings reports, screenshots
```

Architecture

Stack

CSV → PostgreSQL → Node.js/Express → React, as mandated. There is no other database or application framework.

Layer	Technology	Responsibility
Data pipeline	Python 3 (pandas, numpy, psycopg 3)	Land the CSV verbatim, detect and clean anomalies, build the cleaned model, simulate Snowflake `ACCOUNT_USAGE`, run and verify the SQL suite, generate the data-quality and findings docs
Database	PostgreSQL 16 (Docker Compose)	Single application database: raw, cleaned, application state, simulated logs, cost views
API	Node.js + Express 5 (JavaScript, ESM)	REST API, RBAC, sessions, telemetry ingestion, alert lifecycle, WebSocket hub, replay
Real-time	`ws` WebSocket server on the same HTTP server	Cookie-authenticated, permission- and owner-filtered push
UI	React 19 + Vite, React Router, Recharts	Dark-only operations console



Database schemas

Schema	Contents
`raw`	`report_monitoring_csv`: every value as text, with `source_row_number` lineage
`app`	`reports` (100), `report_observations` (2,000), `dq_rules`, `dq_issues`, `users`, `owner_overrides`, `v_report_current`, `audit_log`, `telemetry_events`, `alerts`, `alert_events`
`portal`	`report_usage_monthly` (view over observations), `credit_pricing`
`account_usage`	Simulated `query_history`, `query_attribution_history`, `warehouse_metering_history`
`cost`	Views created by the Snowflake SQL suite (dropped and rebuilt on each run)

Migrations are idempotent SQL files in `db/migrations/`, applied by `pic_pipeline.migrate` and tracked with checksums in `public.schema_migrations`.

The data model behind the CSV

The CSV has no timestamp. The 2,000 rows are 20 complete, identically ordered passes over 100 report ids, which was verified for every row. The pipeline models them as a report **dimension** (100) and an **observation fact** (report × snapshot, 2,000). Snapshot *k* maps to month 2025-02 + (k − 1). The monthly cadence is tested (forensic check 1); the anchor is the last complete month before the data was received (`DATA_AS_OF_MONTH`).

- **Current state** of a report = its latest snapshot.
- **Current owner** = owner in the latest snapshot, or an admin override (`app.owner_overrides`, applied by `app.v_report_current`).
- **Nothing is deleted.** Defects are nulled or quarantined in typed columns, flagged with booleans, and recorded once per (row, rule) in `app.dq_issues`. The raw text always stays in `raw`.

Request path and security

1. `helmet` (CSP, no `x-powered-by`), CORS allowlist with credentials, 256 KiB JSON limit, cookie parser, `pino-http` logging, API latency metrics.
2. CSRF guard: non-GET browser requests require `X-Requested-With: XMLHttpRequest`. Requests carrying `X-API-Key` are exempt.
3. Per-mount guard: `authenticate` (session cookie → user re-read from the DB), then `requirePermission(<perm>)`. Routes that need a narrower permission add a second guard (for example, `alerts:ack` under `alerts:read`).
4. Handlers validate input with zod and apply `ownerScope` for REPORT_OWNER. Owner-scoped misses return 404.
5. A central error handler maps `HttpError` to the documented error envelope. Unknown errors are logged and returned as a generic 500.

Real-time design (Task 2)

- **One ingest path** for the three sources (replay, collectors, browser RUM): persist the telemetry event, evaluate the pure rules (`alertRules.js`), apply the alert lifecycle in the same transaction (`alertService.js`), commit, then broadcast.
- **Sequential processing** through an in-process promise queue, plus a partial unique index, guarantees one live incident per rule and report.
- **Replay reset** truncates alerts and telemetry through the same serial queue as ingestion, and a generation counter discards any batch that was in flight, so a reset can never deadlock with, or be partially undone by, concurrent telemetry.
- **Replay** turns the static dataset into a live feed. It streams the 2,000 observations in snapshot order, 10 batches per snapshot, at a configurable interval. A `telemetry.snapshot` summary is sent after each snapshot.
- **Back-pressure in the UI:** at most 3 toasts on screen, overflow summarised, and a mute toggle. Repeat breaches arrive as silent `alert.updated` messages.
- **Resilience:** heartbeat ping/terminate, client reconnect with jittered exponential backoff, and REST resync with `since`.

Cost allocation (Task 3)

See COST_ALLOCATION_METHODOLOGY.md. The SQL suite in `sql/snowflake/` builds the `cost.*` views that `/api/costs/*` reads. Its six reconciliation checks run in `npm run verify` and in pytest.

Frontend

- `AuthProvider` loads `/api/auth/me`, and `RequirePermission` gates routes. Navigation is derived from the same permission names as the API (`auth/navigation.js`).
- `AlertStreamProvider` owns the socket, the alert reducer (`alertsReducer.js`, pure and tested), toasts, replay state and an event bus that pages subscribe to (`useStreamEvent`).
- `useApi` keeps the previous data while refetching (rendered at reduced opacity) to avoid layout flashes.
- `rum.js` measures how long each page takes to become usable and posts it to `/api/telemetry/rum`, so the Command Center monitors its own dashboard load latency.
- **Theme:** dark-only, set in Geist (Geist Mono for numbers), with design tokens in `theme/tokens.css`. The accent is the Everpure brand orange `#ff7023` (everpuredata.com token `--ps-color-brand-500`; the active navigation tab uses Everpure's brand gradient `#ff7023 → #d55d1d` with white text), and chart marks use the darker brand step `#d55d1d`, which passes the full CVD/contrast validation against the blue second series. The layout is inspired by a two-tone analytics dashboard concept: top pill navigation, large light-weight titles and numbers, rounded charcoal cards for tables and charts, lighter glass cards for KPIs, pill filter groups, gradient bars with a solid cap (`components/chartShapes.jsx`) and a glow callout. The Critical KPI card and critical pop-up notifications use a

red background (white text ≥ 5:1). The severity palette (sage, amber, copper, red `#c63d42`) was validated for colour-vision-deficiency separation, normal-vision separation and ≥ 3:1 contrast on the card surface. Severity always comes with an icon and a text label, and every chart has a table view.

Testing

Suite	Runner	Covers
`pipeline/tests`	pytest	every anomaly count; threshold edges (15.0 / 15.01 / 55.0 / invalid); owner normalisation; fail-fast on bad numerics; simulation reconciles per report-month; determinism; SQL portability guard; full pipeline into the test DB; all reconciliation checks
`server/test`	`node:test` + fetch + `ws`	rule edges; RBAC matrix; 401/403/404 scoping; CSRF; cookie flags; login rate limit; owner reassignment; raise → escalate → de-duplicate → auto-resolve over a real WebSocket; owner-scoped delivery; ack/resolve workflow; API-key ingest; RUM; replay of snapshot 1 (77 alerts); OpenAPI contract
`client/src/test`	Vitest + Testing Library	permission-gated navigation; 403 route for a report owner; login redirect; alert reducer (ordering, stale events, resync merge); backoff; severity never colour-only

Screenshots

All screenshots were captured from the running application on the real dataset, across the four roles. Tall pages are shown from the top; the README has all 15 at full size, with fuller explanations.

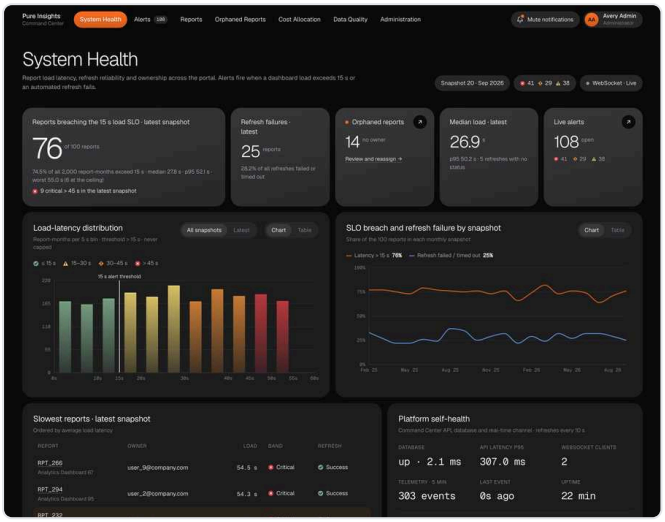


Figure 1. System Health (Task 1). Operator landing page: 76 of 100 reports breach the 15 s load SLO in the latest snapshot (74.5 % of all report-months). Latency histogram with the 15 s threshold, breach and refresh-failure trend, and platform self-health.

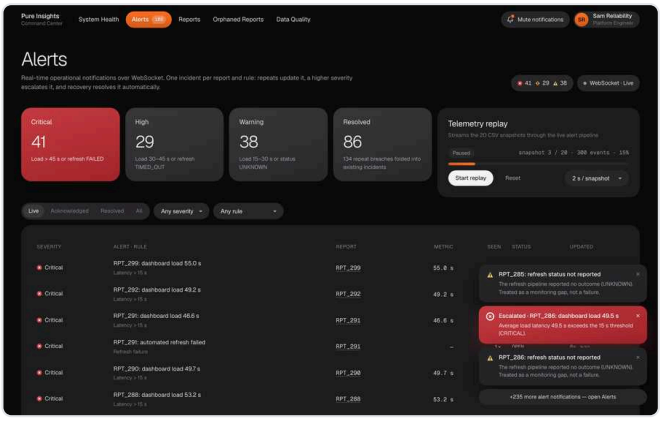


Figure 2. Real-time WebSocket alerting (Task 2). Mid-replay as the Platform Engineer. Alerts arrive over the WebSocket as toasts; repeat breaches fold into one incident per report and rule (108 live incidents, 86 auto-resolved, 134 repeats de-duplicated).

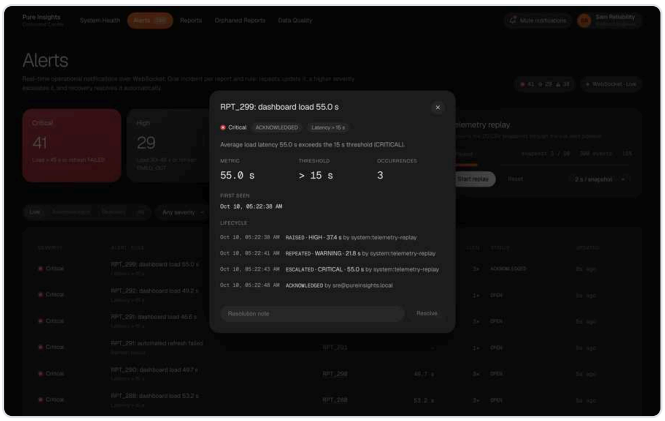


Figure 3. Alert lifecycle (Task 2). RPT_299: raised at HIGH (37.4 s), repeated, escalated to CRITICAL at 55.0 s (the dataset's extreme, alerted on as-is, never capped), then acknowledged. Every transition is stored with its actor.

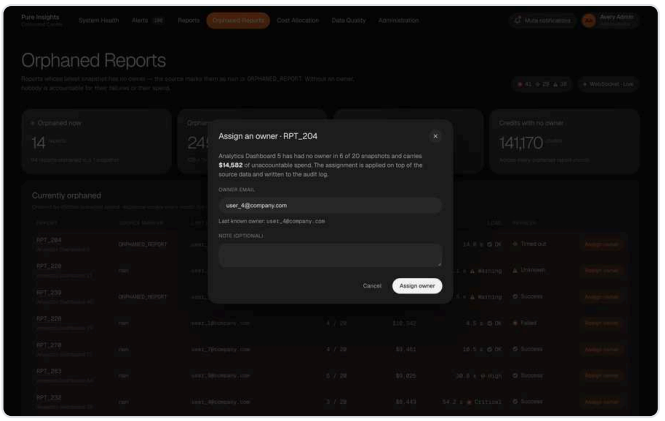


Figure 4. Orphaned reports (Task 1). The 14 reports orphaned in the latest snapshot, with raw markers (nan / ORPHANED_REPORT), last known owner and unowned spend ($480.9K across 245 report-months). Admin-only, audited owner reassignment.

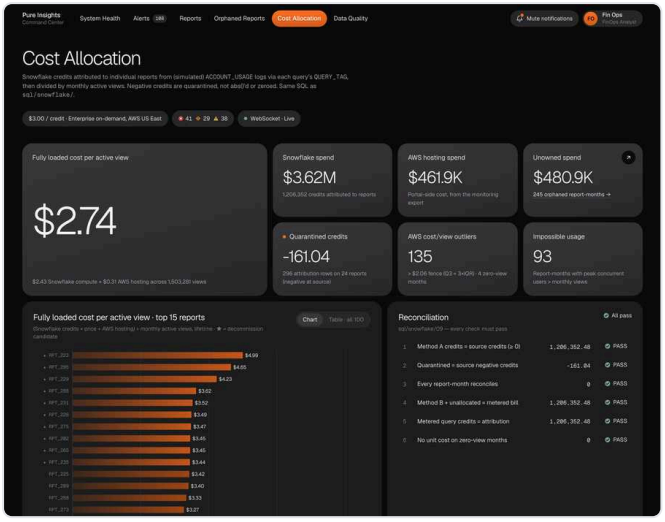


Figure 5. Cost allocation (Task 3). Credits attributed per report from QUERY_ATTRIBUTION_HISTORY via QUERY_TAG, using the SQL suite's cost.* views. $2.74 fully loaded per active view ($2.43 Snowflake + $0.31 AWS); all six reconciliation checks PASS.

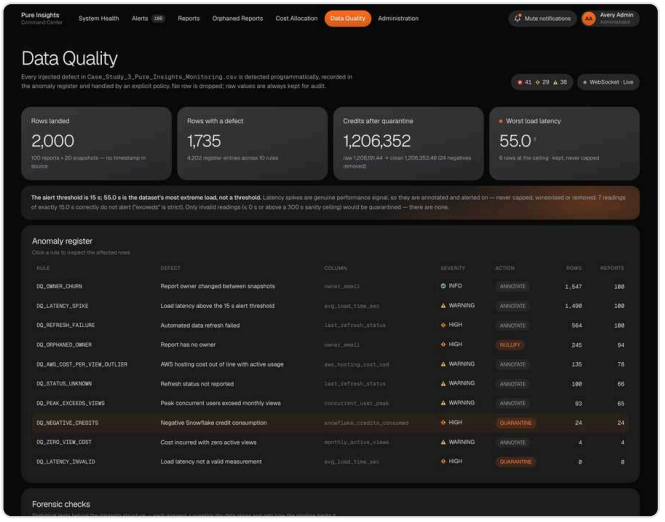


Figure 6. Data quality register (injected anomalies). 10 rules with detection logic, severity, action (NULLIFY / QUARANTINE / ANNOTATE) and affected rows: 245 orphaned owners, 24 negative credits, 1,490 latency breaches, 93 impossible peaks, 135 AWS cost-per-view outliers.

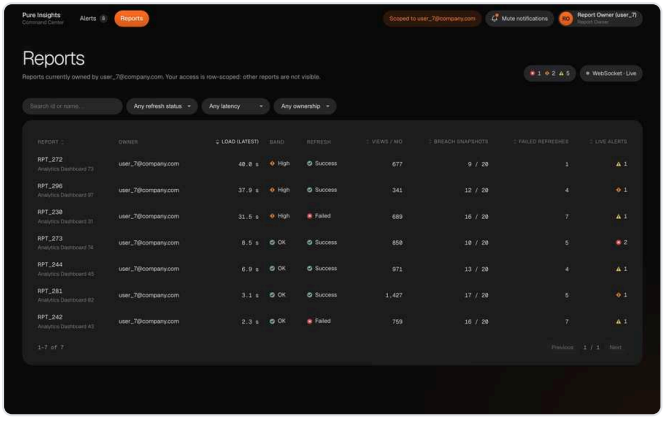


Figure 7. RBAC: row-scoped Report Owner. Signed in as user_7@company.com: only Alerts and Reports in the navigation, and only the 7 reports this user currently owns. Alerts and WebSocket messages are filtered to the same scope on the server.

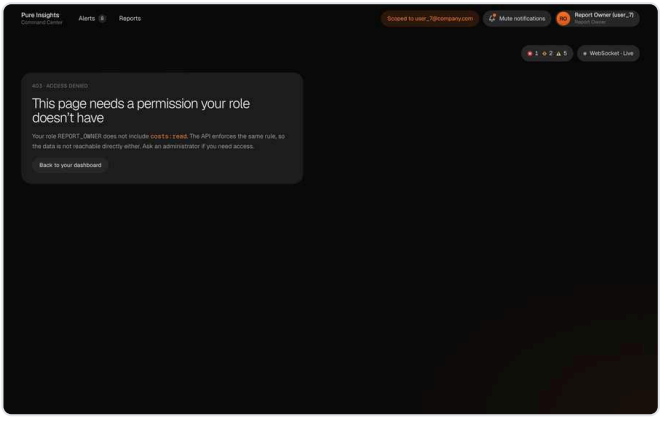


Figure 8. RBAC: permission denied. Opening /costs directly as a Report Owner shows a 403 naming the missing permission (costs:read). The API returns 403 for the same request and records the denial in the audit log.

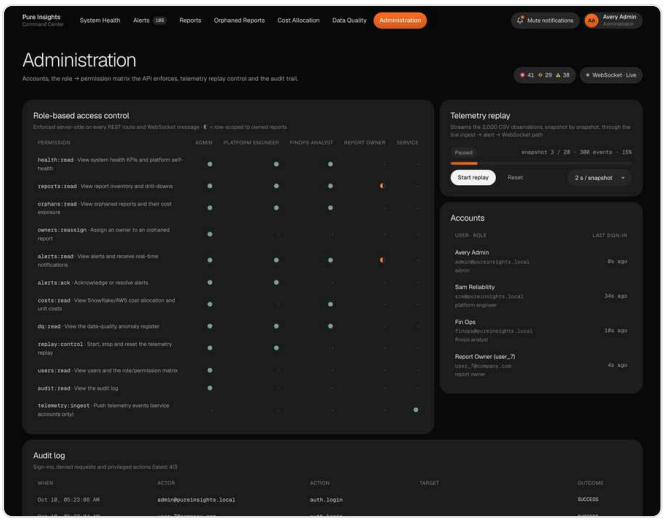


Figure 9. Administration. The role-to-permission matrix the API enforces, replay controls, accounts and the audit trail.

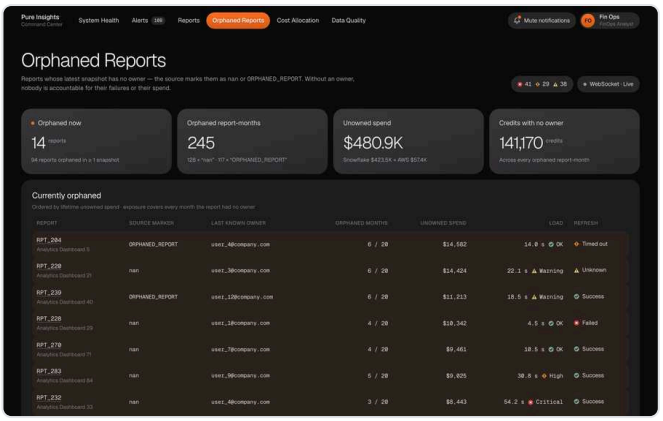


Figure 10. FinOps Analyst: read-only orphan view. FinOps sees the same 14 orphaned reports and unowned spend, without the Assign owner action (owners:reassign is admin-only and enforced by the API).

DELIVERABLE 2

REST API & WebSocket technical specification

Reproduced from `docs/API_SPEC.md`. The machine-readable OpenAPI 3.1 and AsyncAPI 3.0 versions are in the repository.

`docs/API_SPEC.md` `docs/openapi.yaml` `docs/asyncapi.yaml`

Pure Insights Command Center · API version 1.0.0

This document specifies the HTTP API (Node.js/Express) and the real-time WebSocket channel that power the Command Center dashboard. Machine-readable versions:

- `openapi.yaml`: OpenAPI 3.1, covering every REST operation. A contract test (`server/test/openapi.contract.test.js`) fails the build if a mounted Express route is missing from it or if the spec documents a route that does not exist.
- `asyncapi.yaml`: AsyncAPI 3.0, covering every WebSocket message.

1. Conventions

Item	Convention
Base URL	`http://localhost:4000` (in development the Vite server on `:5173` proxies `/api` and `/ws`)
Format	JSON request and response bodies, UTF-8
Naming	Query/body parameters are `camelCase`; response rows mirror database columns (`snake_case`)
Time	ISO-8601 UTC timestamps; report periods are `YYYY-MM-DD` dates (first of month)
Validation	Every input is validated with zod. Failures return `400 BAD_REQUEST` with per-field `details`
SQL	Parameterised queries only; sort columns are chosen from a whitelist

Error model

Every error has the same envelope:

```
{ "error": { "code": "FORBIDDEN", "message": "Missing permission: costs:read", "details": null } }
```

Status	code	When
400	`BAD_REQUEST`, `BAD_JSON`	Validation failure or malformed JSON
401	`UNAUTHENTICATED`, `INVALID_CREDENTIALS`, `INVALID_API_KEY`	No valid session, wrong password, wrong collector key
403	`FORBIDDEN`, `CSRF_CHECK_FAILED`	Role lacks the permission, or a browser request is missing the CSRF header
404	`NOT_FOUND`	Unknown resource, **or** a report outside an owner-scoped user's scope (so existence is not leaked)
409	`INVALID_STATE`	Illegal alert transition (for example, acknowledging an already acknowledged alert)
429	`TOO_MANY_ATTEMPTS`, `RATE_LIMITED`	Login brute-force limit or RUM beacon limit
500	`INTERNAL`	Unexpected error (logged with the request id; no internals leaked)

Pagination

List endpoints that can grow take `page` (1-based) and `pageSize`, and return `{ page, pageSize, total, <items> }`.

2. Authentication & sessions

Login method: local accounts in PostgreSQL (`app.users`), passwords hashed with **bcrypt (cost 12)**. No third-party identity provider is used. An OIDC provider (Okta / Azure AD) could replace the login step behind the same session middleware.

```
POST /api/auth/login        { "email": "sre@pureinsights.local", "password": "…" }
→ 200 { "user": { email, displayName, role, ownerEmail, ownerScoped, permissions[] } }
  Set-Cookie: pic_session=<JWT>; Path=/; HttpOnly; SameSite=Strict; Max-Age=28800  (+ Secure in production)
```

- The session is a **JWT (HS256, 8 h, issuer `pure-insights-command-center`)** stored in an **httpOnly** cookie. The token is never returned in a body or exposed to JavaScript.
- The user row is re-read on every request, so deactivating an account or changing its role takes effect immediately.
- **CSRF:** the cookie is `SameSite=Strict`. In addition, every browser request must send `X-Requested-With: XMLHttpRequest`. A cross-site page cannot add that header without a CORS preflight, and the CORS allowlist rejects that preflight.
- **Brute force:** failed logins are limited to `LOGIN_RATE_LIMIT` (default 10) per IP per 15 minutes. Unknown e-mails still run a bcrypt compare, so response timing does not reveal which accounts exist. Failures are audited.
- `GET /api/auth/me` returns the current user. `POST /api/auth/logout` clears the cookie.
- **Telemetry collectors** (machine-to-machine) send `X-API-Key: <TELEMETRY_API_KEY>`, which is compared in constant time. They get the **SERVICE** role, whose only permission is `telemetry:ingest`.

Demo accounts

Account	Role
`admin@pureinsights.local`	ADMIN
`sre@pureinsights.local`	PLATFORM_ENGINEER
`finops@pureinsights.local`	FINOPS_ANALYST
`user_7@company.com`	REPORT_OWNER (row-scoped to the reports user_7 currently owns)

All demo accounts use the password from `SEED_DEMO_PASSWORD` in `.env`.

3. Role-based access control

The matrix lives in `server/src/auth/permissions.js`. It is enforced on every REST route and filters every WebSocket message. The React app mirrors it for navigation only. Every denial is written to `app.audit_log`.

Permission	ADMIN	PLATFORM_ENGINEER	FINOPS_ANALYST	REPORT_OWNER	SERVICE
`health:read`: system health and platform self-health	●	●	●		
`reports:read`: report inventory and drill-down	●	●	●	◐	
`orphans:read`: orphaned reports and exposure	●	●	●		
`owners:reassign`: assign an owner	●				
`alerts:read`: alerts and real-time notifications	●	●	●	◐	
`alerts:ack`: acknowledge / resolve	●	●			
`costs:read`: Snowflake/AWS cost allocation	●		●		
`dq:read`: data-quality register	●	●	●		
`replay:control`: start/stop/reset replay	●	●			
`users:read`: accounts and RBAC matrix	●				
`audit:read`: audit log	●				
`telemetry:ingest`: push telemetry					●

◐ means **row-scoped**: a REPORT_OWNER sees only reports whose *current* owner (latest snapshot, or an admin override) equals their `owner_email`. This applies to the alerts of those reports too. Out-of-scope reports return 404.

4. REST endpoints

Method	Path	Permission	Purpose
POST	`/api/auth/login`	public	Sign in (sets cookie)
POST	`/api/auth/logout`	public	Clear session
GET	`/api/auth/me`	authenticated	Current user and permissions
GET	`/api/health`	public	Liveness probe (DB round-trip)
GET	`/api/health/system`	`health:read`	DB latency, WebSocket clients, replay, telemetry rate, API p50/p95/p99
GET	`/api/dashboard/summary`	`health:read`	Latest-snapshot and all-time KPIs, live alerts by severity
GET	`/api/dashboard/latency-distribution?scope=all|latest`	`health:read`	5 s histogram with severity band per bin
GET	`/api/dashboard/trend`	`health:read`	Per-snapshot breach rate, failure rate, p50/p95
GET	`/api/dashboard/slowest?limit=`	`health:read`	Slowest reports in the latest snapshot
GET	`/api/reports`	`reports:read` ◐	Filter/sort/paginate the inventory

Method	Path	Permission	Purpose
GET	`/api/reports/{reportId}`	`reports:read` ◐	20-snapshot history, owner timeline, alerts, DQ issues, cost
GET	`/api/orphans`	`orphans:read`	Current orphans with unowned spend, summary, reassignments
GET	`/api/orphans/owners`	`owners:reassign`	Owner picker list
POST	`/api/orphans/{reportId}/assign-owner`	`owners:reassign`	Admin owner override (audited, broadcast)
GET	`/api/alerts`	`alerts:read` ◐	List / resync (`status`, `severity`, `rule`, `reportId`, `since`, `limit`)
GET	`/api/alerts/stats`	`alerts:read` ◐	Counts and last-hour lifecycle events
GET	`/api/alerts/{alertId}`	`alerts:read` ◐	Alert and full lifecycle
POST	`/api/alerts/{alertId}/acknowledge`	`alerts:ack`	OPEN → ACKNOWLEDGED
POST	`/api/alerts/{alertId}/resolve`	`alerts:ack`	Manual resolve with a note
POST	`/api/telemetry/events`	`telemetry:ingest` (API key)	Ingest collector events
POST	`/api/telemetry/rum`	authenticated	Browser page-load beacon
GET	`/api/telemetry/recent`	`health:read`	Latest telemetry events
GET	`/api/telemetry/replay`	`health:read`	Replay status
POST	`/api/telemetry/replay/start` | `stop` | `reset` | `config`	`replay:control`	Replay control (`config` takes `intervalMs`, `loop`, `pauseAfterSnapshot`)
GET	`/api/costs/summary`	`costs:read`	Portfolio unit cost, reconciliation, quarantine, metering, discrepancy, unit cost by edition price
GET	`/api/costs/reports`	`costs:read`	Per-report unit cost, ranking, decommission flag, Method A vs B
GET	`/api/costs/reports/{reportId}`	`costs:read`	Monthly unit cost and Method B allocation
GET	`/api/costs/trend`	`costs:read`	Monthly Snowflake and AWS spend
GET	`/api/costs/chargeback`	`costs:read`	Spend by owner (orphaned spend explicit)
GET	`/api/costs/discrepancy`	`costs:read`	AWS cost vs views, with the 3×IQR fence and flags
GET	`/api/data-quality/summary`	`dq:read`	Totals, credits before/after quarantine, latency extremes
GET	`/api/data-quality/rules`	`dq:read`	Rule catalog with counts
GET	`/api/data-quality/forensics`	`dq:read`	Statistical checks behind the structural findings
GET	`/api/data-quality/issues`	`dq:read`	Anomaly register rows
GET	`/api/admin/users`	`users:read`	Accounts
GET	`/api/admin/rbac`	`users:read`	Role → permission matrix
GET	`/api/admin/audit`	`audit:read`	Audit trail

Example: ingest telemetry and get alert transitions back

```
POST /api/telemetry/events
X-API-Key: pic-dev-telemetry-key
Content-Type: application/json

{ "events": [ { "type": "report.telemetry", "reportId": "RPT_200",
                "avgLoadTimeSec": 52.3, "refreshStatus": "TIMED_OUT" } ] }
```

```
202 Accepted
{ "accepted": 1,
  "transitions": [
    { "type": "alert.raised", "alertId": 41, "ruleCode": "LATENCY_SLO_BREACH", "reportId": "RPT_200",
"severity": "CRITICAL" },
    { "type": "alert.raised", "alertId": 42, "ruleCode": "REFRESH_FAILURE",    "reportId": "RPT_200",
"severity": "HIGH" } ] }
```

Event schema: `type` = `report.telemetry`, `reportId` (`^[A-Za-z0-9_-]{1,40}$`), and at least one of `avgLoadTimeSec` (number) or `refreshStatus` (string). Optional fields: `snapshotSeq` and `observedAt`. A request carries 1–500 events.

Example: owner reassignment

```
POST /api/orphans/RPT_204/assign-owner
X-Requested-With: XMLHttpRequest
{ "ownerEmail": "user_4@company.com", "note": "Last known owner, confirmed by BI lead" }
→ 200 { "report": { "report_id": "RPT_204", "owner_email": "user_4@company.com", "is_orphaned": false,
"has_owner_override": true, … } }
```

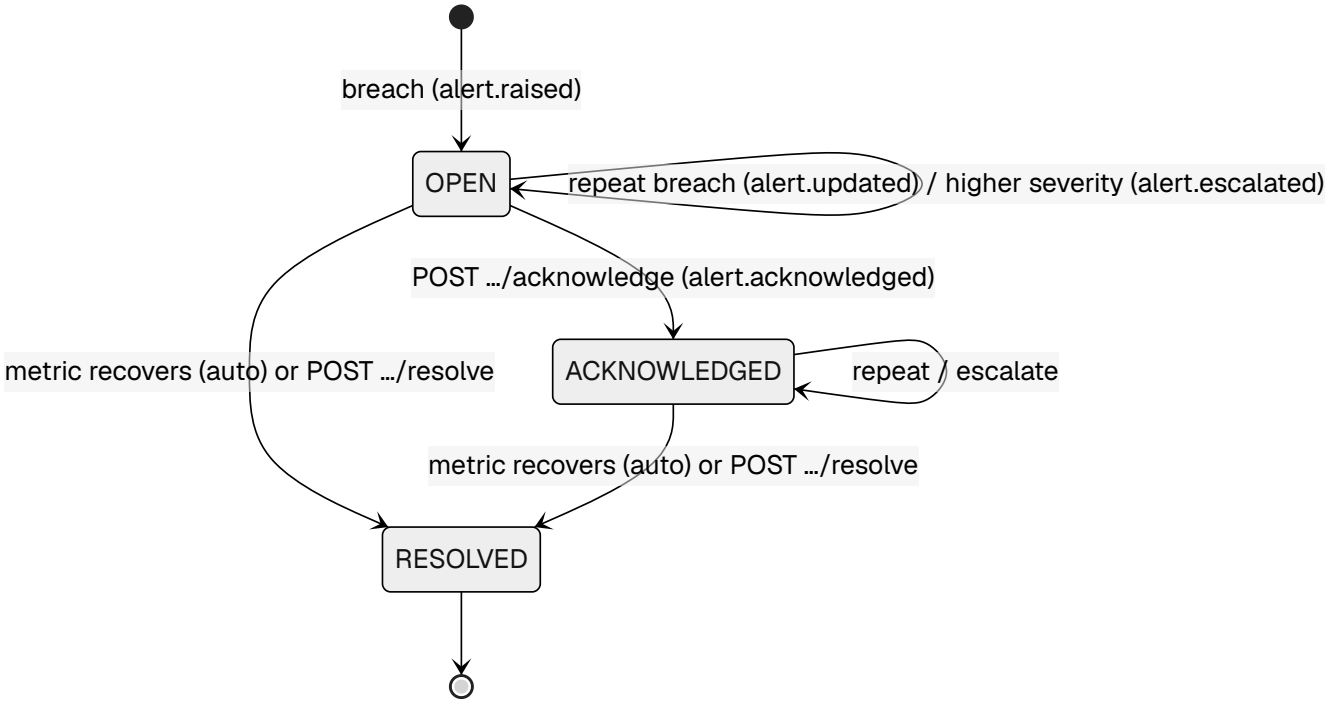
The source data is never modified. The override sits in `app.owner_overrides`, is applied through the `app.v_report_current` view, is written to the audit log, refreshes the WebSocket owner scopes, and is broadcast as `report.owner_changed`.

5. Alerting rules (Task 2)

Rule	Fires when	Severity	Clears when
`LATENCY_SLO_BREACH`	`avg_load_time_sec` **strictly greater than 15 s**	WARNING ≤ 30 s < HIGH ≤ 45 s < CRITICAL	latency ≤ 15 s
`REFRESH_FAILURE`	refresh status `FAILED` / `TIMED_OUT`	CRITICAL / HIGH	status `SUCCESS`
`TELEMETRY_GAP`	refresh status `UNKNOWN` (or unrecognised)	WARNING	any known status
`RUM_SLOW_LOAD`	a Command Center page took > 15 s to load	by latency band	next load ≤ 15 s

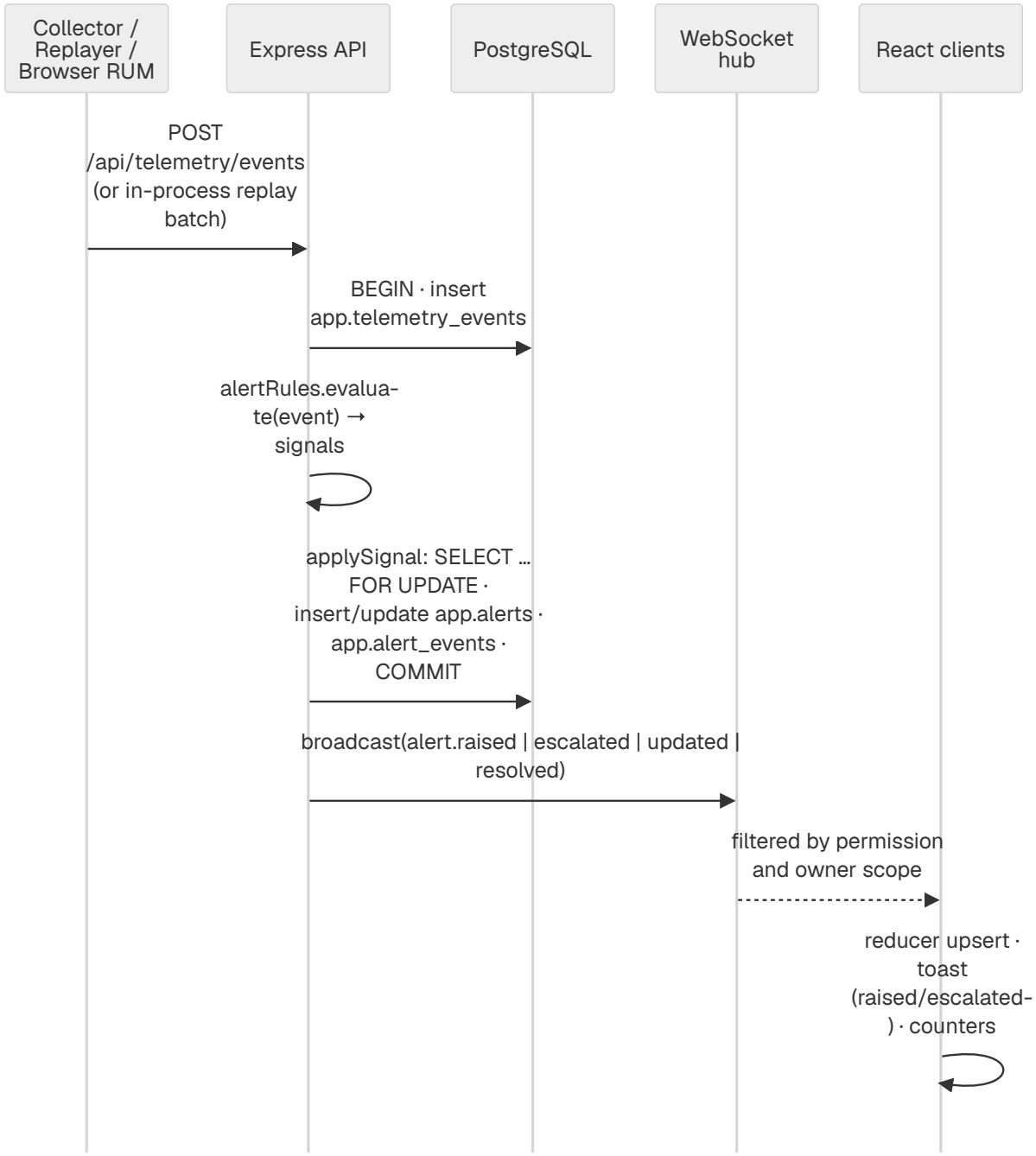
- **15 s is the only threshold taken from the brief.** The 30 s and 45 s tiers are configurable display bands (`LATENCY_HIGH_ABOVE_SEC`, `LATENCY_CRITICAL_ABOVE_SEC`).
- **55 s is not a threshold.** It is the largest load latency in the dataset (6 readings sit exactly at 55.0 s). Spikes are never capped. A 55 s reading raises a CRITICAL alert with `metric_value = 55`.
- Readings that are not valid measurements (non-finite, ≤ 0 s, or > `LATENCY_INVALID_ABOVE_SEC` = 300 s) are stored but never alert. They count as bad data, not bad performance.
- A reading of exactly 15.0 s does **not** alert, because the brief says "exceeds".

Incident lifecycle and de-duplication



- At most **one live incident per `dedupe_key`** (`<rule>:<reportId>`, or `RUM_SLOW_LOAD:<route>`). A partial unique index on `app.alerts (dedupe_key) WHERE status <> 'RESOLVED'` enforces this.
- `severity` is the current severity and `peak_severity` is the worst seen. `occurrence_count` counts breaches folded into the incident.
- Every transition is stored in `app.alert_events` (RAISED / REPEATED / ESCALATED / ACKNOWLEDGED / RESOLVED) with the actor.
- Events are processed sequentially by an in-process queue, so concurrent ingests cannot race the de-duplication.
- Why this matters: 74.5 % of the dataset's report-months exceed 15 s. Without de-duplication the stream would emit about 1,490 notifications instead of tens of incidents.

Data flow



6. WebSocket channel

Handshake

```
GET /ws HTTP/1.1
Upgrade: websocket
Cookie: pic_session=<JWT>
Origin: http://localhost:5173
```

Outcome	Response
Valid session and `alerts:read`	`101 Switching Protocols`, then a `session.ready` message
Missing or invalid cookie	`401 Unauthorized` (upgrade refused)
Origin not allowed, or role lacks `alerts:read`	`403 Forbidden`

Outcome	Response
Path other than `/ws`	`404 Not Found`

The same cookie authenticates REST and WebSocket, so no token ever appears in a URL or in JavaScript.

Envelope

```
{ "type": "alert.escalated", "id": "2f0c3b7e-…", "ts": "2026-10-10T03:38:39.120Z", "data": { … } }
```

`id` is unique per message. Clients use it to de-duplicate toasts.

Messages (server → client)

type	Payload	Delivered to
`session.ready`	`{ email, role, scopedReports }`	the connecting client
`alert.raised`	Alert	`alerts:read`; owner-scoped by `report_id`
`alert.escalated`	Alert (higher severity)	same
`alert.updated`	Alert (repeat breach; count/metric changed)	same
`alert.acknowledged`	Alert	same
`alert.resolved`	Alert (auto-recovery or manual)	same
`alerts.cleared`	`{ reason }`	`alerts:read` (replay reset)
`report.owner_changed`	Report (current view row)	`reports:read`; owner-scoped
`telemetry.snapshot`	`{ snapshotSeq, reports, latencyBreaches, refreshFailures, medianLatencySec }`	`health:read`
`replay.status`	ReplayStatus	`health:read`
`system.heartbeat`	`{ clients }`	everyone, every 30 s

Inbound client messages are ignored (`maxPayload` 16 KiB).

Liveness, reconnect and resync

1. The server sends a protocol ping every 30 s. A socket that misses a pong is terminated.
2. On close, the client reconnects with **exponential backoff and jitter**: delay = random(½, 1) × min(30 s, 1 s × 2^attempt).
3. On every successful (re)connect the client **resyncs over REST**. The first connect calls `GET /api/alerts?status=live`. Later connects call `GET /api/alerts?status=all&since=<serverTime of last sync>` and merge the result. The reducer keeps whichever copy has the newer `updated_at`, so out-of-order events cannot roll state back.
4. The UI shows the connection state (Live / Connecting / Reconnecting / Offline). At most 3 toasts are shown at a time; any burst beyond that becomes a single "+N more" item.

7. Operational limits

Limit	Value
Login failures	`LOGIN_RATE_LIMIT` per IP per 15 min (default 10)
RUM beacons	20 per minute per signed-in user; `route` must be one of the app's route templates; `loadTimeSec` ≤ 600

Limit	Value
Telemetry batch	1–500 events per request
JSON body	256 KiB
WebSocket inbound frame	16 KiB
Session lifetime	`SESSION_TTL_HOURS` (default 8)

DELIVERABLE 3

Snowflake cloud cost allocation SQL script suite

The suite overview from `sql/snowflake/README.md`, then the methodology and results. Every script is listed in full in the appendix.

`sql/snowflake/` `docs/COST_ALLOCATION_METHODOLOGY.md`

Attributes Snowflake credit consumption to individual Pure Insights reports and computes the unit cost per active user view. It runs against simulated `ACCOUNT_USAGE` logs. Methodology, figures and assumptions are in `docs/COST_ALLOCATION_METHODOLOGY.md`.

Script	Creates	Purpose
`00_bootstrap_simulation_snowflake.sql`	DB `PURE_INSIGHTS_SIM`, tables, stage	**Snowflake only.** Loads the simulated logs exported to `data/simulated/*.csv`
`01_source_profiling.sql`	—	Coverage, untagged queries, failed queries, negative attributions
`02_anomaly_quarantine.sql`	`cost.v_query_attribution_classified`, `v_quarantine_summary`, `v_failed_query_spend`	Classifies every credit as VALID / NEGATIVE_CREDITS / UNTAGGED / UNKNOWN_REPORT
`03_attribution_query_tag.sql`	`v_report_month_credits_method_a`, `v_report_credits_method_a`	**Method A**: per-query attribution by `QUERY_TAG`
`04_attribution_metering_proportional.sql`	`v_query_hour_share`, `v_report_month_credits_method_b`, `v_metering_unallocated`, `v_attribution_method_comparison`, `v_warehouse_idle`	**Method B**: hourly metered bill allocated by execution-time share; idle analysis
`05_unit_cost_per_active_view.sql`	`v_unit_cost_report_month`, `v_unit_cost_report`, `v_unit_cost_portfolio`, `v_unit_cost_price_sensitivity`	Snowflake, AWS and fully loaded cost per active view; zero-view and quarantine safe; unit cost at Standard / Enterprise / Business Critical list prices
`06_orphaned_cost_exposure.sql`	`v_orphaned_cost_exposure`, `v_orphaned_cost_summary`	Spend with no accountable owner
`07_aws_cost_vs_usage_discrepancy.sql`	`v_aws_cost_per_view_fence`, `v_aws_usage_discrepancy(_summary)`, `v_aws_usage_correlation`	Usage vs AWS cost discrepancies (IQR fence, zero views, peak > views, Pearson/Spearman)

Script	Creates	Purpose
`08_chargeback_and_ranking.sql`	`v_chargeback_owner(_month)`, `v_report_cost_ranking`, `v_report_cost_mom`, `v_cost_trend_month`	Chargeback, ranking, decommission candidates, month-over-month change
`09_reconciliation_checks.sql`	`v_reconciliation_checks`	Six invariants. Every row must be `PASS`

Portability

Scripts 01–09 use one ANSI subset: CTEs, window functions, `PERCENTILE_CONT … WITHIN GROUP`, `CORR`, `SPLIT_PART`, `NULLIF`, `DATE_TRUNC`, and `CAST(… AS NUMERIC(p,s))`. They avoid PostgreSQL-only syntax (`::`, `FILTER`, `ILIKE`) and Snowflake-only syntax (`QUALIFY`, `IFF`, `FLATTEN`). A pytest guard (`test_suite_is_portable_subset`) fails if any of those tokens appear. Unqualified schemas (`account_usage`, `portal`, `cost`) resolve inside the current database on both engines.

Run on PostgreSQL (this repository)

```
npm run verify        # rebuilds the cost.* views from 01-09 and asserts every reconciliation check
```

Run on Snowflake (simulated data)

```
npm run pipeline      # also exports data/simulated/*.csv
snowsql -f sql/snowflake/00_bootstrap_simulation_snowflake.sql   # after the PUT shown in the script
# then, in PURE_INSIGHTS_SIM with ALTER SESSION SET TIMEZONE = 'UTC':
snowsql -d PURE_INSIGHTS_SIM -f sql/snowflake/01_source_profiling.sql   # … through 09
```

Verified as Snowflake SQL

```
npm run verify:snowflake      # or: cd pipeline && .venv/bin/python -m pic_pipeline.cli snowflake
```

1. **Parse.** Every statement in `00-09` parses as Snowflake SQL with `sqlglot`.
2. **Execute.** The suite runs in Snowflake dialect on fakesnow, a Snowflake emulator backed by DuckDB. `00`'s DDL builds `PURE_INSIGHTS_SIM`, the exported logs are loaded, `01-09` execute, and all 6 reconciliation checks pass with figures identical to PostgreSQL. The emulator has no stages and no `ALTER SESSION`, so `00`'s `PUT`/`COPY INTO` are replaced by a bulk load there. Both are covered by the live run below.
3. **Live.** `python -m pic_pipeline.cli snowflake-live` runs the complete script, including `PUT` and `COPY INTO`, against a real account when `SNOWFLAKE_ACCOUNT`, `SNOWFLAKE_USER` and `SNOWFLAKE_PASSWORD` (optionally `SNOWFLAKE_WAREHOUSE`, `SNOWFLAKE_ROLE`) are set. This needs an account, for example a free trial, and is the only step not yet run.

Run against a live Snowflake account

`SNOWFLAKE.ACCOUNT_USAGE` is read-only, so create a shim schema with the same names in your own database and run 01–09 there:

```
CREATE SCHEMA IF NOT EXISTS ANALYTICS.ACCOUNT_USAGE;
CREATE OR REPLACE VIEW ANALYTICS.ACCOUNT_USAGE.QUERY_HISTORY              AS SELECT * FROM
SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY;
CREATE OR REPLACE VIEW ANALYTICS.ACCOUNT_USAGE.QUERY_ATTRIBUTION_HISTORY  AS SELECT * FROM
SNOWFLAKE.ACCOUNT_USAGE.QUERY_ATTRIBUTION_HISTORY;
CREATE OR REPLACE VIEW ANALYTICS.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY AS SELECT * FROM
SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY;
-- plus PORTAL.REPORT_USAGE_MONTHLY (the portal's usage facts) and PORTAL.CREDIT_PRICING
```

This requires the portal to set `QUERY_TAG = 'pure-insights:report=<REPORT_ID>'` on every dashboard query.

Methodology and results

Goal (from the brief): write SQL against simulated Snowflake `ACCOUNT_USAGE` logs that attributes credit consumption to individual reports, and calculate the unit cost per active user view.

The scripts are in `sql/snowflake/`. The same files run on Snowflake and on PostgreSQL. In this repository they create the `cost.*` views that the Cost Allocation page and `/api/costs/*` read, so the dashboard and the deliverable SQL cannot drift apart.

1. Why the logs are simulated, and how

The CSV has per-report, per-period totals (`snowflake_credits_consumed`, `query_count_last_30d`, `monthly_active_views`, `aws_hosting_cost_usd`). It has **no** warehouse, hour or query detail, but real `ACCOUNT_USAGE` views are at that grain. `pipeline/pic_pipeline/simulate_account_usage.py` therefore generates query-level logs **from** the CSV. It uses a fixed seed, so the output is identical on every run. The column names follow Snowflake's documented views.

Simulated view	Grain	How it is derived
`QUERY_HISTORY`	one row per representative query batch (≈ 1 per 1,000 executions; ≥ 1 per report-month) → 25,230 rows	`QUERY_TAG = 'pure-insights:report=<id>'`; each report pinned to one of 3 warehouses; execution time follows the credits each batch burns at the warehouse's hourly rate. Failed and timed-out refreshes in the CSV leave a failed batch (`000630` statement timeout / `002003` missing object)
`QUERY_ATTRIBUTION_HISTORY`	one row per query	the report-month's CSV credits split across its batches in proportion to execution weight, so they **reconcile exactly** to the CSV. The 24 negative-credit report-months keep their negative values, so the injected defect exists at source
`WAREHOUSE_METERING_HISTORY`	warehouse × hour → 18,999 rows	valid (non-negative) attributed credits booked to the hour each batch starts. `CREDITS_ATTRIBUTED_COMPUTE_QUERIES` excludes idle, as in Snowflake
`PORTAL.REPORT_USAGE_MONTHLY`	report × month (2,000)	the portal's own facts: owner, views, peak users, AWS cost, credits as reported by the legacy export
`PORTAL.CREDIT_PRICING`	one row	`CREDIT_PRICE_USD` = **$3.00**, Snowflake's on-demand list price for Enterprise edition on AWS US East (stand-in for `ORGANIZATION_USAGE.RATE_SHEET_DAILY`)

The simulated layer adds no spend of its own. The CSV carries no evidence of warehouse idle time, untagged ad-hoc work or concurrency effects, so none is invented. Every cost figure therefore traces back to the CSV, and Method A and Method B agree to within ±0.07 % per report. The only difference comes from hours where a quarantined negative-credit batch shares the warehouse. The SQL still measures idle time (`v_warehouse_idle`), untagged spend (the `UNTAGGED` quarantine class) and A/B divergence. `test_suite_measures_idle_untagged_and_method_divergence_when_present` switches those effects on (`SimulationKnobs`: 8 % idle, 3 % untagged, execution-time jitter) and checks the SQL detects all three while every reconciliation check still passes.

2. Attribution

Method A: per-query attribution via `QUERY_TAG` (primary)

```
SELECT SPLIT_PART(query_tag, 'report=', 2) AS report_id,
       CAST(DATE_TRUNC('month', start_time) AS DATE) AS period_month,
       SUM(credits_attributed_compute)
FROM   account_usage.query_attribution_history
WHERE  <classified VALID>          -- see 02_anomaly_quarantine.sql
GROUP  BY 1, 2;
```

`QUERY_ATTRIBUTION_HISTORY` is Snowflake's concurrency-aware, per-query compute attribution. Summing it by the tag the portal stamps on every dashboard query attributes credits to individual reports. It excludes warehouse idle time.

Method B: proportional allocation of the metered bill (cross-check)

This is the classic approach. Each warehouse-hour's `CREDITS_USED_COMPUTE` (the bill, idle included) is split across the queries that ran in that hour by their share of `EXECUTION_TIME`. Untagged work goes to `UNATTRIBUTED`, and metered hours with no queries are reported as unallocated idle. Method B allocates the entire bill but is less precise per report, because execution time is only a proxy for credits under concurrency. On a live account the per-report A vs B gap (shown on the Cost page) measures that imprecision plus each report's share of idle time. On this dataset the gap is ≤ 0.07 %, because the CSV implies neither.

3. Anomaly filtering (before any money is computed)

`02_anomaly_quarantine.sql` classifies every attribution row:

Status	Rule	Rows	Treatment
`VALID`	tagged, known report, credits ≥ 0	24,934	attributed
`NEGATIVE_CREDITS`	credits < 0	296 rows / 24 reports / **−161.04** credits	**quarantined**, not abs()'d (that would invent spend) and not zeroed (that would understate cost per view)
`UNTAGGED`	no portal tag	0 here (600 when switched on in tests)	reported separately as unattributable
`UNKNOWN_REPORT` / `NULL_CREDITS`	tag for a report the portal doesn't know / NULL	0	quarantined

4. Unit cost per active user view

```
snowflake_cost_per_view   = Method-A credits × credit_price ÷ monthly_active_views
aws_cost_per_view         = aws_hosting_cost_usd ÷ monthly_active_views
fully_loaded_cost_per_view = (Method-A credits × credit_price + aws_hosting_cost_usd) ÷
monthly_active_views
```

- **Zero views:** `NULLIF(monthly_active_views, 0)` makes the unit cost NULL, with status `ZERO_VIEWS` (4 report-months). It is never a divide-by-zero and never silently 0.
- **Quarantined credits:** the month's Snowflake unit cost is NULL (`CREDITS_QUARANTINED`, 24 report-months). Lifetime and portfolio ratios exclude those months from **both** the numerator and the denominator.
- Grains: report-month (`v_unit_cost_report_month`), report lifetime (`v_unit_cost_report`), portfolio (`v_unit_cost_portfolio`).

Portfolio result (at $3/credit)	Value
Credits attributed to reports	1,206,352.48
Snowflake cost / active view	**$2.434**
AWS hosting cost / active view	**$0.307**
Fully loaded cost / active view	**$2.740**
Per-report fully loaded range	$1.83 – $4.99 (median $2.72)

Price sensitivity (`v_unit_cost_price_sensitivity`, on-demand list prices, AWS US East):

Edition	$/credit	Snowflake $/view	Fully loaded $/view
Standard	$2.00	$1.623	$1.929
Enterprise *(in use)*	$3.00	$2.434	$2.740
Business Critical	$4.00	$3.245	$3.551

> **Statistical caveat.** Report identity explains none of the variance in this dataset (permutation ANOVA p = 0.14–0.88; see forensic check 2). The per-report range and rankings demonstrate the method; the portfolio figures are robust.

5. Discrepancies between active users and AWS cost (`07_…`)

- Pearson correlation of views vs AWS cost is **−0.027** (Spearman with tie-aware ranks −0.027; −0.083 at report grain). Hosting cost does not follow usage.
- `ZERO_VIEW_COST`: 4 report-months with cost and no views.
- `PEAK_EXCEEDS_VIEWS`: 93 report-months where peak concurrent users exceed monthly views, which is impossible, so the usage counters are unreliable there.
- `COST_PER_VIEW_OUTLIER`: AWS cost/view above **Q3 + 3×IQR = $2.064**. That is 135 report-months. `PERCENTILE_CONT` is the same linear interpolation pandas uses, so SQL and Python agree on all 135.

6. Reconciliation (`09_reconciliation_checks.sql`)

#	Check	Expected	Actual	Status
1	Method A = source credits ≥ 0	1,206,352.48	1,206,352.48	PASS
2	Quarantined = source negatives	−161.04	−161.04	PASS
3	Every report-month reconciles (±0.0001)	0 mismatches	0	PASS
4	Method B + unallocated = metered compute	1,206,352.48	1,206,352.48	PASS
5	Metered query credits = all non-negative attribution	1,206,352.48	1,206,352.48	PASS
6	No unit cost on zero-view months	0	0	PASS

`npm run verify` re-runs the suite on PostgreSQL and fails if any check is not `PASS`. `npm run verify:snowflake` parses every statement as Snowflake SQL and executes the suite in Snowflake dialect on the fakesnow emulator. All six checks pass there with figures identical to PostgreSQL. Both runs are part of the pytest suite. `python -m pic_pipeline.cli snowflake-live` runs the same verification on a real account when `SNOWFLAKE_*` credentials are set.

7. Inputs and how each is grounded

1. **Timeline.** The 20 passes are consecutive monthly snapshots. This is tested: lag-1 autocorrelation of the 30-day metrics is ≈ 0, where rolling daily windows would give ≈ +0.97. They are anchored to the last complete month before the data was received: Feb 2025 → Sep 2026 (`DATA_AS_OF_MONTH`).
2. **Credit price.** $3.00 is Snowflake's on-demand list price for Enterprise edition on AWS US East; Standard is $2.00 and Business Critical $4.00. All three appear in `v_unit_cost_price_sensitivity`.
3. **Simulation structure.** Warehouse assignment, batch sampling and start times only shape the logs; they do not change any reported figure. Idle time, untagged work and jitter are 0 because the CSV carries no evidence of them.
4. **Query tag.** The format `pure-insights:report=<id>` is a portal convention. On a live account it would be set per request (`ALTER SESSION SET QUERY_TAG`).

SYNTHESIS

Key findings and assumptions

`docs/BUSINESS_FINDINGS.md` `docs/DATA_QUALITY.md`

Key findings (summary)

The full write-up, generated from live queries, is in docs/BUSINESS_FINDINGS.md.

1. **The latency SLO is broken across the whole platform.** 74.5 % of report-months exceed 15 s, with a median of 27.8 s. Fixing a handful of slow dashboards will not help, so the fix belongs in shared layers: warehouse sizing, caching, payload size.
2. **28.2 % of refreshes fail or time out.** Latency is the same whether a refresh failed or succeeded, so the two problems have independent causes.
3. **Unowned spend is $480.9K**, the largest "owner" in the chargeback. Mandatory ownership at publish time would close it.
4. **AWS hosting cost is decoupled from usage** (r = −0.027), which points to rightsizing. The decommission ranking demonstrates the method; in this dataset report identity carries no statistical signal, so it should be re-run on production data.
5. **Snowflake dominates unit cost.** At the Enterprise price of $3/credit it is $2.43 of the $2.74 per view ($1.62 to $3.25 across editions).
6. **Failed refreshes sit behind $1.14M of spend:** 564 report-months whose refresh failed or timed out carried 337,131 credits ($1.01M) plus $129K of AWS hosting.

Assumptions

Each former assumption is now either tested against the data or tied to an external source:

- **Timeline.** Monthly cadence is *tested* (lag-1 autocorrelation ≈ 0). Only the calendar anchor is a convention: the last complete month before the data was received, Sep 2026. It's configurable via `DATA_AS_OF_MONTH`.
- **Credit price.** `CREDIT_PRICE_USD = 3.00` is Snowflake's on-demand list price for **Enterprise edition on AWS US East**; Standard is $2 and Business Critical $4. The brief puts the portal on AWS, and Enterprise is the edition that provides the multi-cluster warehouses a portal of this concurrency needs. The SQL suite reports unit cost at all three prices (`cost.v_unit_cost_price_sensitivity`).
- **Simulated ACCOUNT_USAGE logs.** The brief asks for simulated logs. They are derived from the CSV and **add no spend of their own**: the CSV has no evidence of warehouse idle time or untagged work, so none is invented, and failed-refresh spend is reported from the CSV itself. The SQL still measures idle and untagged spend; the tests switch both on to prove it.

- **Snowflake execution.** Every statement in `sql/snowflake/00-09` parses as Snowflake SQL, and the whole suite **executes in Snowflake dialect** on the fakesnow emulator with all 6 reconciliation checks passing and figures identical to PostgreSQL (`python -m pic_pipeline.cli snowflake`, also in the test suite). The same script runs against a real account with `python -m pic_pipeline.cli snowflake-live` once `SNOWFLAKE_ACCOUNT/USER/PASSWORD` are set. That last step is the only one not yet run.
- **Severity tiers.** The 30 s and 45 s tiers are configurable display bands. Only the 15 s threshold comes from the brief.

APPENDIX A

SQL scripts 00–09 in full

Source of `sql/snowflake/` at the commit above. Scripts 01–09 are portable ANSI SQL and run unchanged on Snowflake and PostgreSQL; 00 is the Snowflake-only bootstrap.

`00_bootstrap_simulation_snowflake.sql` view on GitHub

65 lines

```
/*
   00 · Bootstrap the simulated ACCOUNT_USAGE logs in a real Snowflake account
   SNOWFLAKE-ONLY script (PostgreSQL loads the same data via the Python pipeline).

   Prerequisite: 'npm run pipeline' (or 'python -m pic_pipeline.cli export') writes
   data/simulated/*.csv. Upload them with SnowSQL PUT (commands below), then run.

   Scripts 01-09 reference unqualified schemas ACCOUNT_USAGE, PORTAL and COST, so
   they run unchanged inside PURE_INSIGHTS_SIM. To run against your *live* account
   instead, replace step 3 with views over SNOWFLAKE.ACCOUNT_USAGE (see README).
*/

USE ROLE SYSADMIN;
CREATE DATABASE IF NOT EXISTS PURE_INSIGHTS_SIM;
USE DATABASE PURE_INSIGHTS_SIM;
CREATE SCHEMA IF NOT EXISTS ACCOUNT_USAGE;
CREATE SCHEMA IF NOT EXISTS PORTAL;
CREATE SCHEMA IF NOT EXISTS COST;
ALTER SESSION SET TIMEZONE = 'UTC';

-- 1 · File format + stage
CREATE OR REPLACE FILE FORMAT PORTAL.PIC_CSV
  TYPE = CSV SKIP_HEADER = 1 FIELD_OPTIONALLY_ENCLOSED_BY = '"'
  NULL_IF = ('') EMPTY_FIELD_AS_NULL = TRUE;
CREATE OR REPLACE STAGE PORTAL.PIC_STAGE FILE_FORMAT = PORTAL.PIC_CSV;

-- From SnowSQL, in the repository root:
--   PUT file://data/simulated/*.csv @PURE_INSIGHTS_SIM.PORTAL.PIC_STAGE AUTO_COMPRESS=TRUE;

-- 2 · Tables (Snowflake column names; subset of the documented views)
CREATE OR REPLACE TABLE ACCOUNT_USAGE.QUERY_HISTORY (
  QUERY_ID VARCHAR, QUERY_TEXT VARCHAR, DATABASE_NAME VARCHAR, SCHEMA_NAME VARCHAR,
  QUERY_TYPE VARCHAR, SESSION_ID NUMBER, USER_NAME VARCHAR, ROLE_NAME VARCHAR,
  WAREHOUSE_ID NUMBER, WAREHOUSE_NAME VARCHAR, WAREHOUSE_SIZE VARCHAR, WAREHOUSE_TYPE VARCHAR,
  QUERY_TAG VARCHAR, EXECUTION_STATUS VARCHAR, ERROR_CODE VARCHAR, ERROR_MESSAGE VARCHAR,
  START_TIME TIMESTAMP_LTZ, END_TIME TIMESTAMP_LTZ, TOTAL_ELAPSED_TIME NUMBER,
  COMPILATION_TIME NUMBER, EXECUTION_TIME NUMBER, QUEUED_OVERLOAD_TIME NUMBER,
  BYTES_SCANNED NUMBER, ROWS_PRODUCED NUMBER, CREDITS_USED_CLOUD_SERVICES NUMBER(38,9));

CREATE OR REPLACE TABLE ACCOUNT_USAGE.QUERY_ATTRIBUTION_HISTORY (
  QUERY_ID VARCHAR, PARENT_QUERY_ID VARCHAR, ROOT_QUERY_ID VARCHAR, WAREHOUSE_ID NUMBER,
  WAREHOUSE_NAME VARCHAR, QUERY_HASH VARCHAR, QUERY_PARAMETERIZED_HASH VARCHAR, QUERY_TAG VARCHAR,
  USER_NAME VARCHAR, START_TIME TIMESTAMP_LTZ, END_TIME TIMESTAMP_LTZ,
  CREDITS_ATTRIBUTED_COMPUTE NUMBER(38,9), CREDITS_USED_QUERY_ACCELERATION NUMBER(38,9));

CREATE OR REPLACE TABLE ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY (
  START_TIME TIMESTAMP_LTZ, END_TIME TIMESTAMP_LTZ, WAREHOUSE_ID NUMBER, WAREHOUSE_NAME VARCHAR,
  CREDITS_USED NUMBER(38,9), CREDITS_USED_COMPUTE NUMBER(38,9),
  CREDITS_USED_CLOUD_SERVICES NUMBER(38,9), CREDITS_ATTRIBUTED_COMPUTE_QUERIES NUMBER(38,9));

CREATE OR REPLACE TABLE PORTAL.REPORT_USAGE_MONTHLY (
  REPORT_ID VARCHAR, REPORT_NAME VARCHAR, PERIOD_MONTH DATE, SNAPSHOT_SEQ NUMBER,
  OWNER_EMAIL VARCHAR, REFRESH_STATUS VARCHAR, AVG_LOAD_TIME_SEC NUMBER(8,2),
  MONTHLY_ACTIVE_VIEWS NUMBER, CONCURRENT_USER_PEAK NUMBER, QUERY_COUNT_LAST_30D NUMBER,
  AWS_HOSTING_COST_USD NUMBER(12,2), SNOWFLAKE_CREDITS_REPORTED NUMBER(12,2));

CREATE OR REPLACE TABLE PORTAL.CREDIT_PRICING (
  PRICING_ID NUMBER, CURRENCY VARCHAR, CREDIT_PRICE_USD NUMBER(10,4));

-- 3 · Load
COPY INTO ACCOUNT_USAGE.QUERY_HISTORY              FROM @PORTAL.PIC_STAGE/query_history.csv.gz;
COPY INTO ACCOUNT_USAGE.QUERY_ATTRIBUTION_HISTORY  FROM @PORTAL.PIC_STAGE/query_attribution_history.csv.gz;
COPY INTO ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY FROM @PORTAL.PIC_STAGE/warehouse_metering_history.csv.gz;
COPY INTO PORTAL.REPORT_USAGE_MONTHLY              FROM @PORTAL.PIC_STAGE/portal_report_usage_monthly.csv.gz;
COPY INTO PORTAL.CREDIT_PRICING                    FROM @PORTAL.PIC_STAGE/portal_credit_pricing.csv.gz;
```

01_source_profiling.sql view on GitHub

26 lines

```sql
/*
   01 · Source profiling — what do the ACCOUNT_USAGE logs cover?
   Portable: runs unchanged on Snowflake and PostgreSQL.
*/

SELECT 'QUERY_HISTORY'               AS source_view,
       COUNT(*)                      AS row_count,
       MIN(start_time)               AS first_event,
       MAX(start_time)               AS last_event,
       SUM(CASE WHEN query_tag IS NULL OR query_tag = '' THEN 1 ELSE 0 END) AS untagged_rows,
       SUM(CASE WHEN execution_status <> 'SUCCESS' THEN 1 ELSE 0 END)       AS failed_rows
FROM account_usage.query_history
UNION ALL
SELECT 'QUERY_ATTRIBUTION_HISTORY', COUNT(*), MIN(start_time), MAX(start_time),
       SUM(CASE WHEN query_tag IS NULL OR query_tag = '' THEN 1 ELSE 0 END),
       SUM(CASE WHEN credits_attributed_compute < 0 THEN 1 ELSE 0 END)
FROM account_usage.query_attribution_history
UNION ALL
SELECT 'WAREHOUSE_METERING_HISTORY', COUNT(*), MIN(start_time), MAX(start_time), 0, 0
FROM account_usage.warehouse_metering_history
UNION ALL
SELECT 'PORTAL.REPORT_USAGE_MONTHLY', COUNT(*),
       CAST(MIN(period_month) AS TIMESTAMP), CAST(MAX(period_month) AS TIMESTAMP),
       SUM(CASE WHEN owner_email IS NULL THEN 1 ELSE 0 END),
       SUM(CASE WHEN monthly_active_views = 0 THEN 1 ELSE 0 END)
FROM portal.report_usage_monthly;
```

02_anomaly_quarantine.sql view on GitHub

61 lines

```sql
/*
   02 · Anomaly quarantine — classify every attributed credit before it is used
   Credits cannot be negative. The injected negative values are QUARANTINED
   (excluded from attribution, kept for audit) — never abs()'d or zeroed.
   Untagged queries and tags pointing at unknown reports are separated too.
*/

CREATE OR REPLACE VIEW cost.v_query_attribution_classified AS
WITH known_reports AS (
    SELECT DISTINCT report_id FROM portal.report_usage_monthly
),
parsed AS (
    SELECT a.query_id,
           a.warehouse_name,
           a.start_time,
           CAST(DATE_TRUNC('month', a.start_time) AS DATE)                      AS period_month,
           a.query_tag,
           CASE WHEN a.query_tag LIKE 'pure-insights:report=%'
                THEN SPLIT_PART(a.query_tag, 'report=', 2) END                  AS tagged_report_id,
           a.credits_attributed_compute
    FROM account_usage.query_attribution_history a
)
SELECT p.query_id,
       p.warehouse_name,
       p.start_time,
       p.period_month,
       p.query_tag,
       p.tagged_report_id                                                       AS report_id,
       p.credits_attributed_compute,
       CASE WHEN p.credits_attributed_compute IS NULL       THEN 'NULL_CREDITS'
            WHEN p.credits_attributed_compute < 0           THEN 'NEGATIVE_CREDITS'
            WHEN p.tagged_report_id IS NULL                 THEN 'UNTAGGED'
            WHEN k.report_id IS NULL                        THEN 'UNKNOWN_REPORT'
            ELSE 'VALID' END                                                    AS attribution_status
FROM parsed p
LEFT JOIN known_reports k ON k.report_id = p.tagged_report_id;

CREATE OR REPLACE VIEW cost.v_quarantine_summary AS
SELECT attribution_status,
       COUNT(*)                                 AS query_rows,
       COUNT(DISTINCT report_id)                AS reports_affected,
       SUM(credits_attributed_compute)          AS credits,
       MIN(credits_attributed_compute)          AS min_credits,
       MAX(credits_attributed_compute)          AS max_credits
```

```sql
FROM cost.v_query_attribution_classified
GROUP BY attribution_status;

-- Spend burned by refresh queries that failed or timed out (still billed by Snowflake)
CREATE OR REPLACE VIEW cost.v_failed_query_spend AS
SELECT c.report_id,
       c.period_month,
       h.error_code,
       COUNT(*)                                  AS failed_query_rows,
       SUM(c.credits_attributed_compute)         AS credits_on_failed_queries
FROM account_usage.query_history h
JOIN cost.v_query_attribution_classified c ON c.query_id = h.query_id
WHERE h.execution_status <> 'SUCCESS'
  AND c.attribution_status = 'VALID'
GROUP BY c.report_id, c.period_month, h.error_code;

SELECT * FROM cost.v_quarantine_summary ORDER BY attribution_status;
```

03_attribution_query_tag.sql view on GitHub

29 lines

```sql
/*
   03 · METHOD A — per-query attribution via QUERY_TAG (primary method)
   Snowflake's QUERY_ATTRIBUTION_HISTORY assigns warehouse compute credits to
   each query (concurrency-aware, excludes idle time). The portal stamps every
   dashboard query with QUERY_TAG = 'pure-insights:report=<REPORT_ID>', so
   summing VALID rows by tag attributes credits to individual reports.
*/

CREATE OR REPLACE VIEW cost.v_report_month_credits_method_a AS
SELECT report_id,
       period_month,
       COUNT(*)                          AS attributed_query_rows,
       SUM(credits_attributed_compute)   AS credits_attributed
FROM cost.v_query_attribution_classified
WHERE attribution_status = 'VALID'
GROUP BY report_id, period_month;

CREATE OR REPLACE VIEW cost.v_report_credits_method_a AS
SELECT report_id,
       COUNT(*)                          AS months_with_valid_credits,
       SUM(attributed_query_rows)        AS attributed_query_rows,
       SUM(credits_attributed)           AS credits_attributed
FROM cost.v_report_month_credits_method_a
GROUP BY report_id;

SELECT report_id, months_with_valid_credits, credits_attributed
FROM cost.v_report_credits_method_a
ORDER BY credits_attributed DESC
LIMIT 10;
```

04_attribution_metering_proportional.sql view on GitHub

77 lines

```sql
/*
   04 · METHOD B — proportional allocation of the warehouse bill (cross-check)
   The classic approach used before QUERY_ATTRIBUTION_HISTORY existed: take each
   warehouse-hour's metered compute credits (the bill, INCLUDING idle time) and
   split it across the queries that ran in that hour by execution-time share.
   Untagged work is allocated to 'UNATTRIBUTED'. Metered hours with no queries
   are reported as unallocated idle.

   Method B fully allocates the bill; Method A is more precise per query. The
   reconciliation in 09 proves: SUM(B) + unallocated = SUM(metering).
*/

CREATE OR REPLACE VIEW cost.v_query_hour_share AS
WITH q AS (
    SELECT h.query_id,
           h.warehouse_name,
           DATE_TRUNC('hour', h.start_time)                          AS hour_start,
           CAST(DATE_TRUNC('month', h.start_time) AS DATE)           AS period_month,
           CASE WHEN h.query_tag LIKE 'pure-insights:report=%'
                THEN SPLIT_PART(h.query_tag, 'report=', 2)
                ELSE 'UNATTRIBUTED' END                              AS report_id,
           CAST(h.execution_time AS NUMERIC(38,9))                   AS execution_ms
```

```
    FROM account_usage.query_history h
    WHERE h.warehouse_name IS NOT NULL
      AND h.execution_time > 0
)
SELECT q.*,
       q.execution_ms / SUM(q.execution_ms) OVER (PARTITION BY q.warehouse_name, q.hour_start) AS
hour_time_share
FROM q;

CREATE OR REPLACE VIEW cost.v_report_month_credits_method_b AS
SELECT s.report_id,
       s.period_month,
       SUM(s.hour_time_share * m.credits_used_compute)   AS credits_allocated
FROM cost.v_query_hour_share s
JOIN account_usage.warehouse_metering_history m
  ON m.warehouse_name = s.warehouse_name
 AND m.start_time     = s.hour_start
GROUP BY s.report_id, s.period_month;

CREATE OR REPLACE VIEW cost.v_metering_unallocated AS
SELECT m.warehouse_name,
       m.start_time,
       m.credits_used_compute
FROM account_usage.warehouse_metering_history m
WHERE NOT EXISTS (
    SELECT 1 FROM cost.v_query_hour_share s
    WHERE s.warehouse_name = m.warehouse_name AND s.hour_start = m.start_time
);

-- Method A vs Method B per report (B includes a share of idle time, A does not)
CREATE OR REPLACE VIEW cost.v_attribution_method_comparison AS
WITH b AS (
    SELECT report_id, SUM(credits_allocated) AS credits_method_b
    FROM cost.v_report_month_credits_method_b
    GROUP BY report_id
)
SELECT COALESCE(a.report_id, b.report_id)                                  AS report_id,
       a.credits_attributed                                                 AS credits_method_a,
       b.credits_method_b,
       b.credits_method_b - COALESCE(a.credits_attributed, 0)               AS difference_b_minus_a,
       (b.credits_method_b - a.credits_attributed) / NULLIF(a.credits_attributed, 0) AS difference_pct
FROM cost.v_report_credits_method_a a
FULL OUTER JOIN b ON b.report_id = a.report_id;

-- Warehouse idle overhead (metered compute not attributed to any query)
CREATE OR REPLACE VIEW cost.v_warehouse_idle AS
SELECT warehouse_name,
       SUM(credits_used_compute)                                            AS credits_metered,
       SUM(credits_attributed_compute_queries)                              AS credits_attributed_to_queries,
       SUM(credits_used_compute) - SUM(credits_attributed_compute_queries)  AS credits_idle,
       (SUM(credits_used_compute) - SUM(credits_attributed_compute_queries))
           / NULLIF(SUM(credits_used_compute), 0)                           AS idle_share
FROM account_usage.warehouse_metering_history
GROUP BY warehouse_name;

SELECT * FROM cost.v_warehouse_idle ORDER BY warehouse_name;
```

05_unit_cost_per_active_view.sql view on GitHub

103 lines

```
/*
   05 · Unit cost per active user view
   snowflake_cost_per_view = Method-A credits x contracted $/credit
                             / monthly_active_views
   fully_loaded_cost_per_view adds the report's AWS hosting cost.

   Rules:
   * monthly_active_views = 0  → unit cost is undefined (NULL, status ZERO_VIEWS),
     never a divide-by-zero and never silently 0.
   * credits quarantined (negative in source) → NULL, status CREDITS_QUARANTINED;
     lifetime unit costs exclude those months from BOTH numerator and denominator
     so the ratio is not understated.
*/

CREATE OR REPLACE VIEW cost.v_unit_cost_report_month AS
SELECT u.report_id,
       u.report_name,
       u.period_month,
```

```
       u.owner_email,
       u.monthly_active_views,
       u.aws_hosting_cost_usd,
       a.credits_attributed                                                AS snowflake_credits,
       a.credits_attributed * p.credit_price_usd                           AS snowflake_cost_usd,
       a.credits_attributed / NULLIF(u.monthly_active_views, 0)            AS credits_per_view,
       a.credits_attributed * p.credit_price_usd
           / NULLIF(u.monthly_active_views, 0)                             AS snowflake_cost_per_view_usd,
       u.aws_hosting_cost_usd / NULLIF(u.monthly_active_views, 0)          AS aws_cost_per_view_usd,
       CASE WHEN a.credits_attributed IS NULL THEN NULL
            ELSE (a.credits_attributed * p.credit_price_usd + u.aws_hosting_cost_usd)
                 / NULLIF(u.monthly_active_views, 0) END                   AS fully_loaded_cost_per_view_usd,
       CASE WHEN u.monthly_active_views = 0      THEN 'ZERO_VIEWS'
            WHEN a.credits_attributed IS NULL    THEN 'CREDITS_QUARANTINED'
            ELSE 'OK' END                                                  AS unit_cost_status
FROM portal.report_usage_monthly u
CROSS JOIN portal.credit_pricing p
LEFT JOIN cost.v_report_month_credits_method_a a
       ON a.report_id = u.report_id
      AND a.period_month = u.period_month;

CREATE OR REPLACE VIEW cost.v_unit_cost_report AS
WITH agg AS (
    SELECT report_id,
           MAX(report_name)                                                AS report_name,
           COUNT(*)                                                        AS months,
           SUM(CASE WHEN unit_cost_status = 'CREDITS_QUARANTINED' THEN 1 ELSE 0 END) AS months_quarantined,
           SUM(CASE WHEN unit_cost_status = 'ZERO_VIEWS' THEN 1 ELSE 0 END)   AS months_zero_views,
           SUM(monthly_active_views)                                       AS total_views,
           SUM(CASE WHEN snowflake_credits IS NOT NULL THEN monthly_active_views ELSE 0 END) AS
views_with_valid_credits,
           SUM(snowflake_credits)                                          AS snowflake_credits,
           SUM(snowflake_cost_usd)                                         AS snowflake_cost_usd,
           SUM(aws_hosting_cost_usd)                                       AS aws_cost_usd,
           SUM(CASE WHEN snowflake_credits IS NOT NULL THEN aws_hosting_cost_usd ELSE 0 END) AS
aws_cost_valid_months
    FROM cost.v_unit_cost_report_month
    GROUP BY report_id
)
SELECT agg.*,
       snowflake_cost_usd + aws_cost_usd                                       AS total_cost_usd,
       snowflake_cost_usd / NULLIF(views_with_valid_credits, 0)                AS
snowflake_cost_per_view_usd,
       aws_cost_usd / NULLIF(total_views, 0)                                   AS aws_cost_per_view_usd,
       (snowflake_cost_usd + aws_cost_valid_months) / NULLIF(views_with_valid_credits, 0) AS
fully_loaded_cost_per_view_usd
FROM agg;

CREATE OR REPLACE VIEW cost.v_unit_cost_portfolio AS
SELECT COUNT(*)                                                            AS reports,
       SUM(total_views)                                                    AS total_views,
       SUM(snowflake_credits)                                              AS snowflake_credits,
       SUM(snowflake_cost_usd)                                             AS snowflake_cost_usd,
       SUM(aws_cost_usd)                                                   AS aws_cost_usd,
       SUM(snowflake_cost_usd) / NULLIF(SUM(views_with_valid_credits), 0)  AS snowflake_cost_per_view_usd,
       SUM(aws_cost_usd) / NULLIF(SUM(total_views), 0)                     AS aws_cost_per_view_usd,
       (SUM(snowflake_cost_usd) + SUM(aws_cost_valid_months))
           / NULLIF(SUM(views_with_valid_credits), 0)                      AS fully_loaded_cost_per_view_usd
FROM cost.v_unit_cost_report;

-- Price sensitivity: the credit price is a contract term, not data. Recompute the portfolio unit
-- cost at Snowflake's on-demand list prices for AWS US East (N. Virginia).
CREATE OR REPLACE VIEW cost.v_unit_cost_price_sensitivity AS
WITH editions AS (
    SELECT 1 AS ordinal, 'Standard' AS edition, CAST(2.00 AS NUMERIC(10,2)) AS credit_price_usd
    UNION ALL SELECT 2, 'Enterprise', CAST(3.00 AS NUMERIC(10,2))
    UNION ALL SELECT 3, 'Business Critical', CAST(4.00 AS NUMERIC(10,2))
),
portfolio AS (
    SELECT SUM(snowflake_credits)        AS credits,
           SUM(views_with_valid_credits) AS views_valid,
           SUM(aws_cost_valid_months)    AS aws_valid
    FROM cost.v_unit_cost_report
)
SELECT e.ordinal,
       e.edition,
       e.credit_price_usd,
       p.credits * e.credit_price_usd                                      AS snowflake_cost_usd,
       p.credits * e.credit_price_usd / NULLIF(p.views_valid, 0)           AS snowflake_cost_per_view_usd,
       (p.credits * e.credit_price_usd + p.aws_valid) / NULLIF(p.views_valid, 0) AS
fully_loaded_cost_per_view_usd
```

```sql
FROM editions e
CROSS JOIN portfolio p;

SELECT report_id, report_name, total_views, snowflake_credits,
       ROUND(snowflake_cost_per_view_usd, 4)     AS snowflake_usd_per_view,
       ROUND(fully_loaded_cost_per_view_usd, 4)  AS fully_loaded_usd_per_view
FROM cost.v_unit_cost_report
ORDER BY fully_loaded_cost_per_view_usd DESC NULLS LAST
LIMIT 10;
```

06_orphaned_cost_exposure.sql view on GitHub

39 lines

```sql
/*
   06 · Orphaned cost exposure — spend with no accountable owner
   A report-month is orphaned when the portal recorded no valid owner e-mail
   (source markers 'nan' / 'ORPHANED_REPORT' are cleaned to NULL upstream).
   "Currently orphaned" = orphaned in the report's latest period.
*/

CREATE OR REPLACE VIEW cost.v_orphaned_cost_exposure AS
WITH latest AS (
    SELECT report_id,
           owner_email,
           ROW_NUMBER() OVER (PARTITION BY report_id ORDER BY period_month DESC) AS rn
    FROM portal.report_usage_monthly
)
SELECT c.report_id,
       MAX(c.report_name)                                                  AS report_name,
       MAX(CASE WHEN l.owner_email IS NULL THEN 1 ELSE 0 END)              AS is_currently_orphaned,
       COUNT(*)                                                            AS orphaned_months,
       SUM(c.monthly_active_views)                                         AS orphaned_views,
       SUM(c.snowflake_credits)                                            AS orphaned_credits,
       SUM(c.snowflake_cost_usd)                                           AS orphaned_snowflake_cost_usd,
       SUM(c.aws_hosting_cost_usd)                                         AS orphaned_aws_cost_usd,
       COALESCE(SUM(c.snowflake_cost_usd), 0) + SUM(c.aws_hosting_cost_usd) AS orphaned_total_cost_usd
FROM cost.v_unit_cost_report_month c
JOIN latest l ON l.report_id = c.report_id AND l.rn = 1
WHERE c.owner_email IS NULL
GROUP BY c.report_id;

CREATE OR REPLACE VIEW cost.v_orphaned_cost_summary AS
SELECT COUNT(*)                                  AS reports_ever_orphaned,
       SUM(is_currently_orphaned)                AS reports_currently_orphaned,
       SUM(orphaned_months)                      AS orphaned_report_months,
       SUM(orphaned_credits)                     AS orphaned_credits,
       SUM(orphaned_snowflake_cost_usd)          AS orphaned_snowflake_cost_usd,
       SUM(orphaned_aws_cost_usd)                AS orphaned_aws_cost_usd,
       SUM(orphaned_total_cost_usd)              AS orphaned_total_cost_usd
FROM cost.v_orphaned_cost_exposure;

SELECT * FROM cost.v_orphaned_cost_summary;
```

07_aws_cost_vs_usage_discrepancy.sql view on GitHub

70 lines

```sql
/*
   07 · AWS hosting cost vs active usage — discrepancy detection
   Hosting cost should scale with usage. Three rules flag where it does not:
     ZERO_VIEW_COST      cost incurred while monthly_active_views = 0
     PEAK_EXCEEDS_VIEWS  concurrent_user_peak > monthly_active_views (impossible)
     COST_PER_VIEW_OUTLIER  aws cost / view > Q3 + 3 x IQR (robust fence)
   plus Pearson and Spearman (tie-aware rank) correlation of views vs cost.
*/

CREATE OR REPLACE VIEW cost.v_aws_cost_per_view_fence AS
SELECT PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY aws_hosting_cost_usd / monthly_active_views) AS q1,
       PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY aws_hosting_cost_usd / monthly_active_views) AS median,
       PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY aws_hosting_cost_usd / monthly_active_views) AS q3
FROM portal.report_usage_monthly
WHERE monthly_active_views > 0;

CREATE OR REPLACE VIEW cost.v_aws_usage_discrepancy AS
SELECT u.report_id,
       u.report_name,
       u.period_month,
```

```
       u.owner_email,
       u.monthly_active_views,
       u.concurrent_user_peak,
       u.aws_hosting_cost_usd,
       u.aws_hosting_cost_usd / NULLIF(u.monthly_active_views, 0)             AS aws_cost_per_view_usd,
       f.q3 + 3 * (f.q3 - f.q1)                                              AS outlier_fence_usd,
       CASE WHEN u.monthly_active_views = 0 AND u.aws_hosting_cost_usd > 0 THEN 1 ELSE 0 END AS
is_zero_view_cost,
       CASE WHEN u.concurrent_user_peak > u.monthly_active_views THEN 1 ELSE 0 END          AS
is_peak_exceeds_views,
       CASE WHEN u.monthly_active_views > 0
             AND u.aws_hosting_cost_usd / u.monthly_active_views > f.q3 + 3 * (f.q3 - f.q1)
            THEN 1 ELSE 0 END                                                    AS is_cost_per_view_outlier
FROM portal.report_usage_monthly u
CROSS JOIN cost.v_aws_cost_per_view_fence f;

CREATE OR REPLACE VIEW cost.v_aws_usage_discrepancy_summary AS
SELECT SUM(is_zero_view_cost)                                  AS zero_view_cost_rows,
       SUM(is_peak_exceeds_views)                              AS peak_exceeds_views_rows,
       SUM(is_cost_per_view_outlier)                           AS cost_per_view_outlier_rows,
       SUM(CASE WHEN is_cost_per_view_outlier = 1 OR is_zero_view_cost = 1
                THEN aws_hosting_cost_usd ELSE 0 END)          AS aws_cost_in_flagged_rows_usd,
       MAX(outlier_fence_usd)                                  AS outlier_fence_usd
FROM cost.v_aws_usage_discrepancy;

CREATE OR REPLACE VIEW cost.v_aws_usage_correlation AS
WITH ranked AS (
    SELECT monthly_active_views,
           aws_hosting_cost_usd,
           concurrent_user_peak,
           RANK() OVER (ORDER BY monthly_active_views)
             + (COUNT(*) OVER (PARTITION BY monthly_active_views) - 1) / 2.0   AS r_views,
           RANK() OVER (ORDER BY aws_hosting_cost_usd)
             + (COUNT(*) OVER (PARTITION BY aws_hosting_cost_usd) - 1) / 2.0   AS r_cost
    FROM portal.report_usage_monthly
),
per_report AS (
    SELECT report_id, SUM(monthly_active_views) AS views, SUM(aws_hosting_cost_usd) AS cost
    FROM portal.report_usage_monthly
    GROUP BY report_id
)
SELECT 'report-month' AS grain,
       CAST(CORR(monthly_active_views, aws_hosting_cost_usd) AS NUMERIC(10,4)) AS pearson_views_vs_aws_cost,
       CAST(CORR(r_views, r_cost) AS NUMERIC(10,4))                            AS spearman_views_vs_aws_cost,
       CAST(CORR(concurrent_user_peak, aws_hosting_cost_usd) AS NUMERIC(10,4)) AS pearson_peak_users_vs_aws_cost
FROM ranked
UNION ALL
SELECT 'report', CAST(CORR(views, cost) AS NUMERIC(10,4)), NULL, NULL
FROM per_report;

SELECT * FROM cost.v_aws_usage_discrepancy_summary;
SELECT * FROM cost.v_aws_usage_correlation;
```

08_chargeback_and_ranking.sql view on GitHub

71 lines

```
/*
   08 · Chargeback, ranking and trend
*/

-- Monthly chargeback by the owner recorded for that period; orphaned spend is explicit.
CREATE OR REPLACE VIEW cost.v_chargeback_owner_month AS
SELECT period_month,
       COALESCE(owner_email, 'UNOWNED (orphaned)')                  AS cost_owner,
       COUNT(*)                                                      AS reports,
       SUM(monthly_active_views)                                     AS views,
       SUM(snowflake_credits)                                        AS snowflake_credits,
       SUM(snowflake_cost_usd)                                       AS snowflake_cost_usd,
       SUM(aws_hosting_cost_usd)                                     AS aws_cost_usd,
       COALESCE(SUM(snowflake_cost_usd), 0) + SUM(aws_hosting_cost_usd) AS total_cost_usd
FROM cost.v_unit_cost_report_month
GROUP BY period_month, COALESCE(owner_email, 'UNOWNED (orphaned)');

CREATE OR REPLACE VIEW cost.v_chargeback_owner AS
SELECT cost_owner,
       SUM(views)              AS views,
       SUM(snowflake_credits)  AS snowflake_credits,
       SUM(snowflake_cost_usd) AS snowflake_cost_usd,
```

```
       SUM(aws_cost_usd)        AS aws_cost_usd,
       SUM(total_cost_usd)      AS total_cost_usd
FROM cost.v_chargeback_owner_month
GROUP BY cost_owner;

-- Lifetime ranking; decommission candidates = top-decile cost per view AND below-median usage
CREATE OR REPLACE VIEW cost.v_report_cost_ranking AS
WITH r AS (
    SELECT u.*,
           RANK() OVER (ORDER BY total_cost_usd DESC)                                AS rank_total_cost,
           RANK() OVER (ORDER BY fully_loaded_cost_per_view_usd DESC)                AS rank_cost_per_view,
           NTILE(10) OVER (ORDER BY fully_loaded_cost_per_view_usd DESC)             AS cost_per_view_decile
    FROM cost.v_unit_cost_report u
),
m AS (
    SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY total_views) AS median_views
    FROM cost.v_unit_cost_report
)
SELECT r.*,
       CASE WHEN r.cost_per_view_decile = 1 AND r.total_views < m.median_views THEN 1 ELSE 0 END
           AS is_decommission_candidate
FROM r CROSS JOIN m;

-- Month-over-month Snowflake spend per report
CREATE OR REPLACE VIEW cost.v_report_cost_mom AS
SELECT report_id,
       period_month,
       snowflake_cost_usd,
       LAG(snowflake_cost_usd) OVER (PARTITION BY report_id ORDER BY period_month)   AS prev_snowflake_cost_usd,
       snowflake_cost_usd
           - LAG(snowflake_cost_usd) OVER (PARTITION BY report_id ORDER BY period_month) AS mom_change_usd
FROM cost.v_unit_cost_report_month;

-- Portfolio trend by month
CREATE OR REPLACE VIEW cost.v_cost_trend_month AS
SELECT period_month,
       SUM(monthly_active_views)                                          AS views,
       SUM(snowflake_credits)                                             AS snowflake_credits,
       SUM(snowflake_cost_usd)                                            AS snowflake_cost_usd,
       SUM(aws_hosting_cost_usd)                                          AS aws_cost_usd,
       SUM(snowflake_cost_usd)
           / NULLIF(SUM(CASE WHEN snowflake_credits IS NOT NULL THEN monthly_active_views ELSE 0 END), 0)
                                                                          AS snowflake_cost_per_view_usd
FROM cost.v_unit_cost_report_month
GROUP BY period_month;

SELECT cost_owner, ROUND(total_cost_usd, 2) AS total_cost_usd
FROM cost.v_chargeback_owner
ORDER BY total_cost_usd DESC;
```

09_reconciliation_checks.sql view on GitHub

69 lines

```
/*
   09 · Reconciliation checks — every row must report status = 'PASS'
   Proves the attribution neither loses nor invents credits:
     1  Method A (valid) = credits the source reported as >= 0
     2  Quarantined negatives = credits the source reported as < 0
     3  Every report-month with valid reported credits reconciles within 0.0001
     4  Method B + unallocated idle = total metered compute (bill fully allocated)
     5  Metering's query-attributed credits = all non-negative attributions
     6  No unit cost computed for a zero-view month
*/

CREATE OR REPLACE VIEW cost.v_reconciliation_checks AS
WITH
src AS (
    SELECT SUM(CASE WHEN snowflake_credits_reported >= 0 THEN snowflake_credits_reported ELSE 0 END) AS
reported_valid,
           SUM(CASE WHEN snowflake_credits_reported <  0 THEN snowflake_credits_reported ELSE 0 END) AS
reported_negative
    FROM portal.report_usage_monthly
),
a AS (
    SELECT SUM(CASE WHEN attribution_status = 'VALID'            THEN credits_attributed_compute ELSE 0 END) AS
a_valid,
           SUM(CASE WHEN attribution_status = 'NEGATIVE_CREDITS' THEN credits_attributed_compute ELSE 0 END) AS
a_negative,
```

```
           SUM(CASE WHEN credits_attributed_compute >= 0          THEN credits_attributed_compute ELSE 0 END) AS
a_non_negative
    FROM cost.v_query_attribution_classified
),
rm AS (
    SELECT COUNT(*) AS mismatched_report_months
    FROM portal.report_usage_monthly u
    LEFT JOIN cost.v_report_month_credits_method_a m
           ON m.report_id = u.report_id AND m.period_month = u.period_month
    WHERE u.snowflake_credits_reported >= 0
      AND ABS(COALESCE(m.credits_attributed, 0) - u.snowflake_credits_reported) > 0.0001
),
b AS (SELECT SUM(credits_allocated) AS b_total FROM cost.v_report_month_credits_method_b),
un AS (SELECT COALESCE(SUM(credits_used_compute), 0) AS unallocated FROM cost.v_metering_unallocated),
mt AS (
    SELECT SUM(credits_used_compute)               AS metered,
           SUM(credits_attributed_compute_queries) AS metered_attributed
    FROM account_usage.warehouse_metering_history
),
z AS (
    SELECT COUNT(*) AS zero_view_unit_costs
    FROM cost.v_unit_cost_report_month
    WHERE monthly_active_views = 0
      AND (snowflake_cost_per_view_usd IS NOT NULL OR fully_loaded_cost_per_view_usd IS NOT NULL)
),
checks AS (
    SELECT 1 AS check_no, 'METHOD_A_EQUALS_SOURCE_VALID_CREDITS' AS check_name,
           src.reported_valid AS expected, a.a_valid AS actual FROM src CROSS JOIN a
    UNION ALL
    SELECT 2, 'QUARANTINE_EQUALS_SOURCE_NEGATIVE_CREDITS', src.reported_negative, a.a_negative FROM src CROSS
JOIN a
    UNION ALL
    SELECT 3, 'REPORT_MONTHS_RECONCILE', 0, rm.mismatched_report_months FROM rm
    UNION ALL
    SELECT 4, 'METHOD_B_PLUS_UNALLOCATED_EQUALS_METERED', mt.metered, b.b_total + un.unallocated
    FROM mt CROSS JOIN b CROSS JOIN un
    UNION ALL
    SELECT 5, 'METERED_QUERY_CREDITS_EQUAL_ATTRIBUTION', mt.metered_attributed, a.a_non_negative FROM mt CROSS
JOIN a
    UNION ALL
    SELECT 6, 'NO_UNIT_COST_ON_ZERO_VIEWS', 0, z.zero_view_unit_costs FROM z
)
SELECT check_no,
       check_name,
       CAST(expected AS NUMERIC(20,4))              AS expected,
       CAST(actual AS NUMERIC(20,4))                AS actual,
       CAST(actual - expected AS NUMERIC(20,6))     AS difference,
       CASE WHEN ABS(actual - expected) <= 0.001 THEN 'PASS' ELSE 'FAIL' END AS status
FROM checks;

SELECT * FROM cost.v_reconciliation_checks ORDER BY check_no;
```